CISSP Domain 8｜ソフトウェア開発セキュリティ 統合マスター学習ノート

8.1 ソフトウェア開発ライフサイクル（SDLC）にセキュリティを理解・統合する

1. 学習目的

教材読み上げの要旨

- 安全なコーディング標準、ガイドライン、フレームワークおよびアーキテクチャ概念の価値と使い方を説明する。
- ソフトウェア開発・管理の方法論を区別し、組織に適切な開発プラクティスの選択を支援する。

補足（試験の考え方）

CISSPの観点では「安全性をテスト工程で最後に付け足す」のではなく、要件定義、設計、実装、テスト、運用・保守、廃止に至るまで安全性を扱う。機能やコストだけでなく、リスク、事業目的、説明責任を判断する。

2. 概要：脆弱性はどこで作り込まれるか

教材読み上げの要旨

- ソフトウェアベースのシステムの安全性は、計画・開発・運用を通じて欠かせない要素として扱う。
- 悪用可能な脆弱性の多くは、要件・設計・ソースコード段階の誤りとして生じる。
- テスト、検証、評価で見逃すと、攻撃者に発見・悪用される状態が残る。

補足：各工程における取り組み例

工程	ありがちな問題	代表的なセキュリティ活動
要件定義	認証・監査・プライバシー要件の漏れ	セキュリティ要件定義、リスク評価、法規制確認
設計	不適切な信頼境界、過剰な権限	アーキテクチャレビュー、脅威モデリング、設計原則の確認
実装	入力検証不足、危険なAPI利用	セキュアコーディング、コードレビュー、SAST
テスト・評価	認証回避や異常系の見逃し	DAST、脆弱性診断、ペネトレーションテスト、受入基準検証
運用・保守	依存ライブラリの脆弱性放置	パッチ・構成・変更・脆弱性管理、継続監視

※上の表は読み上げ本文の列挙ではなく、理解のための補足。

3. 設計・構築・開発を通じたセキュリティ確保：2つのシナリオ

教材読み上げの要旨

1. **新規プロジェクト**：ソフトウェアを構成要素とする新しいシステムを立ち上げる。企画段階から安全な設計・開発・運用を組み込む必要がある。
2. **既存システムの見直し**：既存のソフトウェア群について、有効なセキュリティプロセスがなければ、何を再設計・再開発・交換すべきか判断できない。

どちらも、セキュリティ専門家は開発プロセスと成果物の安全性を支援・確認する。新規開発だけでなく**メンテナンスや変更もセキュリティ改善の機会であり責任**となる。

補足：具体例

状況	セキュリティ専門家の関わり方
新規決済アプリの開発	脅威分析、認証・権限制御・ログの要件確認、設計レビュー
古い業務アプリの刷新	既存脆弱性と依存関係の把握、改善と置換の費用対効果、移行リスク評価

4. リングモデルとOSI 7層モデルの関係・脆弱性が存在する場所

教材読み上げの要旨

- ハードウェアにはファームウェア等の低レベルソフトウェアが搭載される。
- OSはデバイスドライバによって異なるハードウェアの制御・利用を提供し、アプリケーションに詳細を直接意識させずに済ませる。
- OSIモデルの下位層が物理的な実装の詳細を隠蔽し、上位層へ機能を提供する構造との**類似**が説明されている。
- 複数レイヤー・複数システム上のソフトウェアが連携してユーザーの業務機能を実現するため、脆弱性もさまざまなレイヤーや境界に存在し得る。

補足：重要な注意と比較

リングモデルとOSI 7層モデルは同一の階層ではない。 共通点は、抽象化・階層分離・下位機能を上位へ提供する考え方。

モデル	主眼	代表的な整理
保護リング（Ring）モデル	CPU・OSの特権レベルとアクセス保護	典型例：Ring 0＝カーネル特権、Ring 3＝ユーザーモード（CPU/OSにより実装は異なる）
OSI 7層	通信機能の論理的な分割	L1物理、L2データリンク、L3ネットワーク、L4トランスポート、L5セッション、L6プレゼンテーション、L7アプリケーション

脆弱性例：ファームウェアの欠陥、ドライバの特権昇格、OS権限設定ミス、ネットワーク設定ミス、Webアプリの入力検証不足、境界間での信頼の誤認。

試験上の注意：リングの「0」とOSIの「L1/L2」は対応関係を示す数字ではない。

5. アーキテクチャ／脅威モデリング／プロセスレビュー

教材読み上げの要旨

- **脅威モデリング**：潜在的な脅威や不足している保護策を特定・分類・分析するプロセス。OWASPなどが指針を提供する。
- システムをモデル化・抽象化し、サブシステム間および外部環境との**信頼境界**を特定する。
- ソフトウェア、ハードウェア、データ、人が境界をまたぐ箇所を把握する。
- 攻撃者の視点から悪用され得る弱点を検討し、脅威と緩和策を対応づける。
- **STRIDE**を例に、システムをコンポーネントへ分解し、それぞれの脅威と低減策を検討する。ほかに**PASTA、LINDDUN、Trike**が言及された（最後の名称は読み上げ音声からの解釈）。

補足：取り組みの例

1. 資産・業務目的とセキュリティ要件を定める。
2. データフロー図（DFD）を作り、プロセス、データストア、外部主体、データの流れ、信頼境界を描く。
3. 各コンポーネントと境界をSTRIDE等で点検する。
4. 悪用シナリオ、影響、既存管理策を評価し、リスクに応じて緩和策を決める。
5. 設計変更・追加実装後に再評価し、開発ライフサイクル中に更新する。

STRIDE：6種類の脅威

頭文字	英語	日本語	具体例と対策の例
S	Spoofing	なりすまし	他人のアカウントでログイン → 強固な認証、MFA
T	Tampering	改ざん	APIリクエストを書き換える → 入力検証、完全性検証
R	Repudiation	否認	操作したことを否定する → 監査ログ、信頼できる記録
I	Information Disclosure	情報漏えい	権限外データを閲覧する → アクセス制御、暗号化
D	Denial of Service	サービス拒否	大量リクエストでサービス停止 → レート制限、冗長化
E	Elevation of Privilege	権限昇格	一般利用者が管理者機能を実行 → 最小権限、認可検証

※STRIDEは**脅威の分類**であり、上表の管理策は一例。リスク評価やセキュリティテストそのものの代替ではない。

関連する脅威モデリング手法の位置づけ（補足）

手法	特徴・重点
STRIDE	システム・コンポーネントごとに6カテゴリーの脅威を列挙する
PASTA（Process for Attack Simulation and Threat Analysis）	ビジネス目標・リスクと攻撃シナリオを結び、段階的に脅威分析する
LINDDUN	Linkability、Identifiability、Non-repudiation、Detectability、Disclosure of information、Unawareness、Non-compliance：プライバシー脅威の分析に重点
Trike	リスクと許可された操作／アクターの関係を重視した脅威モデリング手法

試験の判断軸：「どう作ると安全か」を設計段階で検討し、**信頼境界・データフロー・脅威・緩和策**を紐づける。STRIDE＝セキュリティ脅威の典型的な分類、LINDDUN＝プライバシーへの重点という違いを押さえる。

6. この時点の暗記ポイント

- セキュリティはSDLCの**全期間で組み込む**。
- 脆弱性の多くは**要件・設計・実装**で作り込まれ、検証不足で残る。
- **新規開発と既存システムの改修・保守**の両方でセキュア開発を考える。
- **リングモデルは特権・保護、OSIは通信機能の分割**。似ているのは階層化・抽象化の考え方。
- 脅威モデリングは**システム分解 → 信頼境界把握 → 脅威洗い出し → 緩和策設計**。
- STRIDE＝**なりすまし、改ざん、否認、情報漏えい、サービス拒否、権限昇格**。

8. 実行可能コンテンツとモバイルコード

教材読み上げの要旨

- Webベースアプリケーションで実行可能コンテンツ／モバイルコードを利用すると、システム構造が拡張され、攻撃対象領域（Attack Surface）が広がる。
- 教材では JavaScript、Javaアーカイブ（JAR）、XML、API が関連項目として挙がった。
- ボットについても関連する自動実行の仕組みとして説明される。
- 多くのマルウェアは、コードを配布・実行する既存の仕組みを悪用する。

補足：用語の正確な区別

用語	本来の役割	セキュリティ上の着眼点
実行可能コンテンツ（Active/Executable Content）	受信先・クライアント等で実行されるコンテンツ	信頼できないコードの実行、XSS、権限・サンドボックス境界
モバイルコード（Mobile Code）	ネットワークを越えて取得・転送され、別の実行環境で動くコード	供給元の信頼性、署名、実行権限、ブラウザー等の分離
JavaScript	Webページ等で動作するスクリプト言語	XSS、悪意ある外部スクリプト、DOM操作、CSPの活用
JAR（Java Archive）	Javaのクラス・リソース等をまとめるアーカイブ形式	中身に実行可能コードを含むことがあるが、JAR自体は言語ではない
XML	データを構造化して表現するマークアップ言語	**XML自体は通常、実行可能コードではない。** XXE、不適切なパーサー設定等に注意
API（Application Programming Interface）	ソフトウェア間の機能・データの呼び出し口	**API自体はモバイルコードではない。**認証・認可、入力検証、レート制限、公開範囲が重要
ボット（Bot）	自動化されたタスクを実行するプログラム	正規のクローラー／チャットボットも悪意あるボットもある。すべてがモバイルコードとは限らない

試験ポイント： 教材に列挙されている関連技術を、そのまま全て「モバイルコードの一種」と同一視しない。特に**JavaScript＝言語、JAR＝配布形式、XML＝データ形式、API＝インターフェース。**

具体的な防御策（補足）

許可されたスクリプトのみを読み込むCSP、入力に応じた適切な出力エンコード、サードパーティのコード審査、最小権限、依存コンポーネント管理、APIの強固な認証・認可、実行環境の分離を検討する。悪性コードだけでなく、正規機能を悪用する攻撃にも注意。

9. ソフトウェアの構築・利用サイクル全体にわたる脆弱性

教材読み上げの要旨

- 設計、コーディング、テスト、配備、利用の各段階に人間のミスや判断の抜けが入り得る。
- **要件の段階から、後の設計・実装に影響するセキュリティ上の欠陥が始まる。**
- セキュリティ機能をソースコードに組み込まない、十分なテスト・検証を行わない、デプロイや利用時の不備を放置することによってシステム全体が危険になる。
- それぞれの段階は、攻撃対象領域とリスクを減らす改善の機会でもある。

補足：工程と対策の対応

工程	脆弱性を作り込む例	実施すべき対策例
要件	認可や監査ログの要件を定めない	セキュリティ要件・悪用ケース・受入基準を定義
設計	信頼境界の見落とし、単一障害点	脅威モデリング、設計レビュー
実装	SQLインジェクション、ハードコードされた秘密	セキュアコーディング、コードレビュー、SAST、秘密情報検査
テスト	正常系だけを確認	異常系・権限テスト、DAST、ペンテスト、受入検証
デプロイ	テスト用設定やデバッグ機能を公開	セキュア設定、変更承認、構成検査
使用・運用	パッチ放置、特権濫用、設定ドリフト	脆弱性管理、監視、適切な権限制御、継続的改修

重要： 早期に発見できるほど、欠陥の修正が開発後期や稼働後に波及しにくい。ただし「早期なら常に安価」と保証するものではなく、リスクや変更範囲による。

10. ソフトウェア開発・利用環境：COTSとシステムライフサイクル

教材読み上げの要旨

- **COTS（Commercial Off-The-Shelf）** が取り上げられる。
- 設計・開発中は脆弱性が作り込まれ、テスト・アセスメントで発見・修正されなければ稼働環境に残る。
- 運用・使用段階では実ユーザーや周辺システムとの相互作用が増え、脅威への露出とリスク管理の難度が高まる。
- 新バージョンの導入・既存バージョンの廃止・ローカルなカスタマイズが発生する。脆弱性が見つかった場合は、**最新版だけでなく利用中の旧バージョンも調査対象**になる。

補足：COTSの定義と選択上の注意

COTS＝既成の商用ソフトウェア製品。 自社でゼロから開発するものではなく、事業者が一般市場向けに提供する既製の製品を調達して利用する。

利点	リスク・対処ポイント
開発期間の短縮、実績・サポートの活用	ソースコードが非公開の場合、内部実装の検証が限られる
標準化された機能や更新	ベンダーの脆弱性対応・サポート期限・パッチ提供に依存
初期開発コストを抑え得る	不要機能、初期設定、連携API、カスタマイズで新たなリスク

採用時はセキュリティ要件、サプライヤー評価、契約・SLA、更新体制、サポート終了（EOL）、データ取扱い、検証可能性を確認する。**購入したからセキュリティ責任がなくなるわけではない。**

補足：ライフサイクル別の特徴

段階	リスクの特徴	管理の中心
設計・開発	問題を作り込み得るが、構造・要件を変更しやすい	設計レビュー、セキュリティテスト、修正
配備・稼働	実利用者・外部システム・攻撃者に晒され、影響範囲が拡大	構成管理、監視、変更・パッチ管理
バージョン更新・廃止	旧版の脆弱性や移行に伴う漏れが残る	全利用版の把握、移行計画、EOL対応、安全な廃止

注意： 脅威・攻撃対象領域は運用時に必ず数学的に最大になるという普遍的法則ではない。教材は「運用環境では接続・利用・利用者が広がり、管理が難しい」という趣旨として理解する。

11. 致命的なトレードオフ：開発時間とエラーの影響

教材読み上げの要旨

- 競争上の優位性のために、組織は新機能や変更機能を素早く提供したい。
- 一方で、目的と要件を十分に記述し、詳細なシステム仕様やビジネスロジック、制約へ落とし込むことが安全性の確認に役立つ。
- その仕様をもとに品質テスト、セキュリティアセスメント、利用者のワークフローやトレーニングを作成できる。
- **安全性を合理的に確認してから公開するのか、1日複数回のような迅速な更新を優先するのか**、という両立上の課題が提示されている。

補足：CISSPでの判断

これは「遅い開発か危険な高速開発か」という二択ではない。リスクに応じた統制を自動化・標準化し、品質とデリバリー速度を両立させる設計が重要。

スピード重視で起こり得る問題	両立に向けたコントロール
レビュー省略・テスト不足	CI/CDで自動テスト・SAST・SCA・秘密検査を組み込む
リリース後の障害	段階的リリース、ロールバック、監視、Feature Flag

スピード重視で起こり得る問題	両立に向けたコントロール
仕様の曖昧さ	セキュリティ受入基準、脅威モデル、追跡可能な要件
重大リスクの見逃し	リスクに応じた人間の承認、例外管理、リスク受容の権限明確化

最重要： 事業価値・セキュリティ・時間・コストのバランスを組織のリスク許容度に基づいて判断する。

12. シンプルなウォーターフォール型SDLCモデル

教材読み上げの要旨

従来型のウォーターフォールは、先行工程で要件・設計を具体化し、後工程へ順番に引き継ぐ開発モデル。読み上げられた工程は次のとおり。

1. コンセプト・ミッションニーズの特定
2. 要件定義
3. システム設計
4. ソフトウェアおよびデータシステムのコーディング
5. 単体テスト（Unit Test）
6. サブシステムテスト（Subsystem/Integration Test）
7. システムテスト（System Test）
8. 受け入れテスト（Acceptance Test）
9. 運用上の使用への配備（Deployment）

教材の論点： セキュリティ、信頼性、安全性などの非機能要件が極めて重要な場合、事前に十分な分析・仕様定義を行う方式が有益なことがある。

補足：特徴と適用判断

観点	ウォーターフォール
強み	要件・設計・承認・成果物の追跡性を確保しやすい。規制・品質保証が厳格な環境に適合しやすい
弱み	変更やフィードバックが後半に集中すると手戻りが大きい。要件の早期固定が困難な案件に不利
セキュリティ上の注意	**工程順で進むことだけでは安全を保証しない。**各工程にレビュー・テスト・是正・変更管理を組み込む
他方式との比較	アジャイル等は反復的な開発・フィードバックを重視するが、それ自体がセキュア／非セキュアを決めるわけではない

テストレベルの区別（補足）

テスト	確認対象	例
単体	関数・クラスなど最小単位	認証トークン検証関数の異常系
サブシステム／統合	モジュール・サービス間の連携	API→認証基盤→DB連携の権限検証

テスト	確認対象	例
システム	システム全体の要件	全体のアクセス制御、ログ、耐障害性
受け入れ	業務・ユーザーの受入基準	発注者が業務要件とセキュリティ要件の充足を確認

13. 今回の直前暗記・ひっかけ整理

設問の語句	思い出すポイント
JavaScript / JAR / XML / API	**言語／アーカイブ／データ形式／インターフェース**。全部が同種のモバイルコードではない
COTS	**既製の商用ソフト**。供給元のパッチ・サポート・契約と残余リスクを評価
初期要件の誤り	設計・実装・テスト・運用まで波及する。**上流からセキュリティを考える**
旧版も稼働中	影響評価は**すべての現役バージョン**に対して実施
開発速度と安全性	自動化・リスクベースの承認・継続検証で両立を図る
ウォーターフォール	要件→設計→実装→段階的テスト→配備。事前仕様化と追跡性が特徴
単体 vs 統合 vs システム vs 受け入れ	**部品／連携／全体／業務要件**を検証

14. 段階ごとのビジネスへの影響と変更コスト【2026-10-27追記】

教材読み上げ内容

- ソフトウェア開発では、**後続の工程へ進んでからの変更・欠陥修正ほど、手戻りやプロジェクトコストが大きくなる傾向**がある。
- 要件・設計の問題を開発後半や運用開始後に発見すると、実装だけでなく設計、テスト、展開、利用者対応のやり直しが生じる。

補足：なぜコストが増えるか

発見段階	必要となり得る対応	具体例
要件定義	要件の修正・関係者合意	「顧客データの保存期間」を要件に追加
設計	アーキテクチャ変更・再レビュー	テナント間のデータ分離方式を設計変更
実装	コード変更・単体テスト	入力検証の欠陥を修正
テスト	修正に加え、統合・回帰テストの再実施	API認可の欠陥により一連のシナリオを再検証
本番運用	修正・試験・変更承認・配備・顧客対応・事故対応等	認可不備により情報漏洩が生じ、緊急改修と調査が必要

注意： これは一般的な傾向で、すべての不具合について後工程ほど必ず何倍高い、という一定の倍率があるわけではない。反復開発・自動テスト・変更容易性によって差は変わる。

CISSP判断軸： 早期のセキュリティ要件定義、設計レビュー、脅威モデリング、段階的テストによって後工程のリスクと手戻りを抑える（Shift Leftの考え方にも通じる）。

15. 「どの程度の安全性・品質が十分か」——ビジネスとミッションのトレードオフ【追記】

教材読み上げ内容

- 観測した欠陥・エラーについて、**修正が必須か、残余リスクを許容できるか**は自動的に決まらない。
- セキュリティと安全性を開発の初期からどの程度組み込むかは、**経営陣・組織のリーダーが事業目的とリスクを踏まえて判断すべき事項**。
- 開発遅延や修正の費用を、リスク低減・移転・回避・受容にかかる費用と混同しない。これらは相互に関係し得るが、同一概念ではない。

補足：リスクに基づく意思決定

対応	意味	例
低減（Mitigate）	発生可能性や影響を小さくする	認可ロジックの改修、セキュリティテストの追加
回避（Avoid）	リスクを生む活動をやめる	高リスクの機能を公開しない
移転／共有（Transfer/Share）	損失負担等を契約・保険等で分担	サイバー保険、契約上の責任分担（説明責任は消えない）
受容（Accept）	評価した残余リスクを所定の権限で引き受ける	軽微な表示不具合を記録して次期改修に回す

例： 医療機器の制御不具合と社内ポータルの表示崩れでは、許容できる影響が違う。どちらも「不具合」だが、人命・規制・契約・可用性・機密性・完全性などを考慮し、事業側の適切な権限者が対応方針を決める。

試験での注意： セキュリティ担当は技術的リスクを評価し対策を提案するが、**組織のリスク受容を独断で決めるのではない**。法令上・安全上の必須要件は、単純な費用対効果だけで放棄できない。

16. 高信頼・高安全システムを設計する実践【追記】

教材読み上げ内容

- 原子力産業のような、失敗時の代償が極めて大きい分野では、**厳密なシステムエンジニアリング**と、ウォーターフォール型に代表される事前計画・レビューを重視した開発の考え方が利用される。
- 早い段階でリスクを発見・排除することが、高い信頼性や安全性の実現につながるという説明。
- 教材では「Six Nines（99.9999%）」のような非常に高い信頼性の例に言及。

補足：理解の際の注意

- **Six Nines＝99.9999%** は、一般に可用性・信頼性等の達成水準を示す表現だが、**何を何時間で測った数字か**を定めなければ評価できない。可用性指標であれば、概算の停止許容時間は年間約31.5秒（365日換算）。

これは教材の特定産業の実績を検証した数値ではなく、算術的な説明例。

- **高可用性（止まりにくさ）、安全性（人への危害を防ぐこと）、セキュリティ（CIA等の保護）は別概念。** 99.9999%の可用性だからセキュア、とはいえない。
- 原子力事故の報告件数が少ないことのみで、特定の開発方式の優位性を因果的に証明できるわけではない。安全文化、冗長化、独立した検証、運用規制等も影響する。
- 要件の追跡可能性、危険分析、形式手法、独立した検証・妥当性確認、多層防御などを組み合わせる考え方が重要。**ウォーターフォールを選べば自動的に高信頼になるわけではない。**

17. 古典的だが根強いソフトウェア設計・コーディングエラー【追記】

教材読み上げ内容と具体例

教材で挙げられたエラー	起こること	理解用の具体例・補足
ループ・条件文の不適切／不完全な処理	終了条件、分岐、境界値を誤る	件数ゼロや最終要素で想定外の処理をする。無限ループ、境界外アクセス等につながる場合がある
データ型の不一致・誤用	値の解釈や計算に誤り	整数の桁あふれ、符号有無の変換、文字列と数値の取り違え
別のプログラム要素によるデータ・コードの悪用	要素間の信頼境界やメモリ保護を破る	共有メモリの不正変更、適切に隔離されていないデータへのアクセス
特殊条件・例外・エラーの検出と処理不足	異常時の安全な動作が欠ける	認証基盤の障害時にアクセスを誤って許可、例外処理漏れによる停止
要素間で受け渡すパラメータの欠落・不正確さ	API・関数間で前提条件が崩れる	必須パラメータ不足、型・形式・意味の検証不足
許容範囲外、欠損、形式上は範囲内でも不正確な入力データ	誤処理・異常動作・誤った結果	生年月日欄に実在しない日付を入力、想定外の長さの文字列、改変された識別子

補足： 上記はカテゴリを理解するための例であり、すべての実装言語・状況で同じ脆弱性につながるとは限らない。特に入力検証は「形式・範囲」だけでなく、**業務上の意味・整合性（semantic validation）**も必要になる。

Garbage In, Garbage Out（GIGO：ガベージイン、ガベージアウト）

教材の主張： 入力データに品質管理がなければ、システムが正常に処理しても、不正確な結果や誤った業務判断が生じ得る。

例： 売上予測モデルへ誤った販売数を投入すれば、計算プログラムに脆弱性がなくても、出力される予測は誤る。

混同注意： GIGOは**データ品質の問題を説明する原則**であり、SQLインジェクションのような特定の攻撃手法名ではない。入力検証・データ品質管理・例外処理は相互に関係するが役割は異なる。

18. 良いプラクティス：言語・統合開発環境の進歩【追記】

教材読み上げ内容

- 古典的なエラーが繰り返されてきたことへの対応として、**プログラミング言語**および**統合開発環境（IDE）**の改善が進んだ。

補足：何が改善に役立つか

技術・習慣	防げる・見つけやすくなる問題	限界
型チェック、より安全な言語機能	型の不一致、不適切なメモリアクセスの一部	誤ったビジネスロジックは残る
IDEの静的解析・Lint・警告	よくあるコードミスや不審パターン	偽陽性・見逃しがある
入力検証と適切な出力エンコード	不正データ処理、注入攻撃の一部	実装文脈ごとの対策が必要
例外処理・Fail Secure設計	エラー時の不安全な動作	要件と動作条件を正しく定義する必要
コードレビュー・自動テスト	不具合の早期検知、変更後の回帰防止	テストされていない条件は残る

CISSP観点： 「どのツールを買うか」より、**設計段階から安全な要件・開発規約・レビュー・検証を組み込む統制**を優先して考える。

19. 今回の暗記トリガー（追記）

問題のキーワード	着眼点
後工程で欠陥が見つかる	手戻り、統合テスト再実施、リリース・運用影響。**早期の発見・修正**
どの程度の安全性でよいか	**事業影響・リスク許容度・法的義務・権限者による判断**
安全性と出荷速度の両立	リスクに応じたゲート、要件追跡、早期検証
原子力・安全重要システム	厳密なエンジニアリング、早期リスク分析、独立検証
Six Nines	**99.9999%**。何の指標か明確にし、セキュリティ保証と混同しない
不適切なループ・条件	境界条件と制御フローの誤り
データ型不一致	型・変換・あふれ等
不正確な入力による不正確な出力	**GIGO**、入力品質と検証
IDE・新しい言語の役割	一部の古典的ミスの予防・早期発見を支援（万能ではない）

20. データ中心の脆弱性コントロールとデータ中心脅威モデリング

教材読み上げ内容

- 不正・想定外の**入力データ**がアプリケーションの異常動作につながり、**不正アクセス、任意コード実行、保存データの破損、異常終了**等を招く。虚偽・不正確なデータが後続処理へ流入するリスクもある。
- ファイルシステム、DB、ナレッジ管理システムに対するユーティリティ等の不適切な使用は、**データ流出**につながり得る。
- 不安全な動作を防ぐには、設計時から有効な**制御ロジック**を組み込む必要がある。教材では市場・規制・法律の圧力とデータ侵害コスト、既存システムの規模に起因する安全性の課題も言及。

補足：NIST SP 800-154（ドラフト）とデータ中心脅威モデリング

- 音声の「8154」は **NIST SP 800-154** を指すと考えられる。文書名は *Guide to Data-Centric System Threat Modeling*（ドラフト）。
- 通常のシステム全体・攻撃面中心の分析と異なり、**守るべき重要データ**を起点として、所在・データフロー・アクセス主体・攻撃経路・統制を追跡する。
- 例：顧客データが入力フォーム→API→DB→バックアップ→出力ファイルへ進む各地点に、入力検証、認証認可、暗号化、最小権限、監査等を適用する。
- **STRIDE**は脅威の種類を体系的に検討する枠組み、**データ中心脅威モデリング**は保護対象データから分析範囲を設定する考え方。併用できる。
- 「PSASS」という音声表現の正式名称は確認できないため断定しない。教材原文を確認するまでは「制御ロジック不備への注意」として扱う。
- IBM Security 2020 *Cost of a Data Breach Report* は教材で挙がった**2020年当時の事例**。現在の業種別コストを示す数値とは区別する。

21. 従来型ウォーターフォールSDLC：工程とセキュリティ活動

21.1 全体像

SDLC（Software Development Life Cycle）：企画・要件・設計・開発・検証・導入・運用保守・廃棄まで、ソフトウェアを管理する一連の段階。**ウォーターフォール**は各段階の成果物を確定・承認しながら順次進める従来型の方法論の一つ。

順	読み上げたフェーズ	主なセキュリティ活動（教材）
1	プロジェクト開始・計画	ユーザー要件から**セキュリティ要件を決定**。代替案に対し**リスク分析**。選択・承認する方針に**セキュリティ戦略**を定義
2	機能要件定義	セキュリティ領域を特定し、**機能上のセキュリティ要件・テスト基準・RFP/契約条件・機能ベースライン**に反映。リスク分析とコンティンジェンシープランも検討
3	システム設計仕様	システムアーキテクチャ、入力・データフロー・出力、インターフェースを設計。**セキュリティ仕様を確立**し、セキュリティテスト計画を更新、**セキュリティベースラインを文書化**
4	開発・実装	設計どおりの安全な実装。**コーディング標準、慎重に選定・検証した信頼できるライブラリ、コード再利用のルール**で反復的なミスを低減
5	文書化・プログラム制御	仕様、操作方法、制御、変更履歴等を文書化し、レビュー可能にする（読み上げでは工程名のみ）
6	テスト・評価、認証・認定、受け入れ	セキュリティ機能と要件適合を検証。**IV&V**や独立したテストチーム、**OT&E**による運用上の評価
7	実運用環境への移行・配備	エンドユーザー組織が主体。**セキュリティ認定、教育・訓練・アウェアネス**などの導入条件を満たす
8	運用・保守・サポート	ハード・ソフトの**構成ベースラインの定期評価・監査**、正式な構成管理、修正・改善・機能追加のレビューと承認
9	廃棄・交換	システムの廃止・後継システムへの交換（今回は工程名のみ言及）

注意： 上表は今回の教材読み上げに沿った段階整理。SDLCの工程名・細分化は教材や組織で異なる。「運用環境への移行は従来型SDLCの一部とみなさない」という教材上の整理も記録するが、**一般的なSDLCの定義では展開・運用もライフサイクルに含むことがある**。

21.2 開始・計画：セキュリティはここから始まる

開発の活動	対応するセキュリティ活動
ユーザー要件の確立	セキュリティ要件の決定
代替案の特定	リスク分析の実施
アプローチの選択・承認	セキュリティ戦略の定義

21.3 機能要件：要件・調達・テストの条件を明示

開発の活動	対応するセキュリティ活動
プロジェクト計画の策定	セキュリティ領域の特定
機能要件の特定	セキュリティ要件の確立
テスト基準の設定	セキュリティテスト戦略・基準の定義
RFP／契約の準備	**セキュリティ要件をRFP・契約に含める**
機能ベースラインの策定	**機能セキュリティ要件**を含める

補足：**RFP（Request for Proposal）**は提案依頼書。外部調達時にもセキュリティ要件を後付けせず、選定・契約条件に含めることが重要。

21.4 設計仕様：セキュリティを設計成果物に落とす

開発の活動	対応するセキュリティ活動
詳細設計の準備	セキュリティ仕様の確立
テスト目標・計画の更新	セキュリティテスト計画の更新
正式ベースラインの策定	セキュリティベースラインの文書化

設計で把握する対象：入力データ／データフロー／出力／システム間インターフェース／アーキテクチャ。前工程で確立した要件を設計仕様にトレースする。

21.5 開発・実装：コードの品質を組織的に管理

教材の重点：**コーディング規約の採用・遵守、検証済みで信頼できるライブラリの選択、コードの再利用に関する明確なルール**。個々のプログラマーの注意力だけに依存しない。

補足：ライブラリの「信頼できる」は無条件の安全を意味しない。依存関係・バージョン・脆弱性・更新方針も継続管理する。

21.6 受け入れ・独立検証：IV&V と OT&E

用語	意味	試験での焦点
Verification（検証）	**仕様どおりに作ったか**（Are we building the product right?）	要件・設計との適合、レビュー、テスト
Validation（妥当性確認）	**目的・ユーザーニーズに合うものを作ったか**（Are we building the right product?）	実利用における適合性
IV&V（Independent Verification and Validation）	開発主体から独立した立場で検証と妥当性確認を行う	**独立性・客観性**。独立テストチームを活用する場合も
OT&E（Operational Test and Evaluation）	実際の運用条件に近い環境で、利用者・顧客側の視点から能力を評価する活動	**運用で本当に使えるか**。大規模・複雑なシステムの顧客主導テストの例

理解用の例： 認証機能が仕様書のパスワード条件を満たす＝Verification。その認証方式が実際の運用・利用者の必要性に合うか＝Validation。独立した第三者チームが評価＝IV&V。現場で負荷や利用手順を含め動作を試す＝OT&E。

21.7 本番移行・運用・改訂

- **本番移行**：受け入れ後、主として利用組織が実施。教材では、セキュリティ認定、実装手順、利用者へのトレーニング・アウェアネス・教育を含む。
- **運用・保守**：実運用開始後もハードウェア・ソフトウェアの**ベースラインを定期評価・監査**する。
- **改訂・交換**：バグ修正、軽微な改善、新機能追加も正式な**構成管理・変更レビュー・承認**の対象。必要に応じセキュリティレビューを行う。
- 補足：セキュリティ「認証・認定（certification/accreditation等）」の具体的な権限者・手続きは教材・制度によって異なるので、いずれか一つの制度と機械的に同一視しない。

21.8 CISSP直前の識別ポイント

設問キーワード	答えの軸
プロジェクト開始前／初期計画	**セキュリティ要件とリスク分析**
提案依頼・契約	**RFP・契約にセキュリティ要求を盛り込む**
詳細設計・インターフェース	**セキュリティ仕様・データフロー・設計ベースライン**
繰返されるコーディングミス	**規約・安全なライブラリ・再利用ルール**
仕様通りか／目的に合うか	**Verification／Validation**
開発チームと独立して評価	**IV&V**
実運用に近い利用者主導テスト	**OT&E**
本番後の軽微な機能変更	**構成管理・変更承認・セキュリティレビュー**

22. 運用・保守フェーズの追加整理【2026-10-27 読み上げ】

教材読み上げ

実運用中は、パフォーマンスの定期監視だけでなく、運用継続性の確保が必要。重要コンポーネントの冗長化、欠陥・弱点の発見と是正、バックアップと復旧手順のテスト、データ・レポートの適切な管理、セキュリティコントロールとプロセスの有効性の確認を行う。重要な変更が生じた場合はリスク再評価や必要に応じた再認定を実施し、SLA・契約遵守を検証する。

CISSPの判断：稼働後も継続監視、構成・変更管理、復旧可能性の検証、リスクに応じた再評価が必要。バックアップを取っただけで満足せず、**復元テスト**まで実施する。

23. 学際的手法：IPTとIPPD

用語	定義・特徴	典型的な利用場面	セキュリティ／CISSPのポイント
IPT（Integrated Product Team：統合製品チーム）	開発・運用・品質・セキュリティ・業務など異なる専門領域の関係者が共通目標に向け協働する**チーム**	大規模・複雑・複数分野が絡む製品やシステム	セキュリティ担当を初期から参画させ、分断による見落としを減らす
IPPD（Integrated Product and Process Development：統合製品・プロセス開発）	製品設計だけでなく開発・製造・試験・検証・維持の**プロセスも一体的に検討するアプローチ**。IPTの活用を推進	要求・製造・試験方式が相互に影響する複雑な開発	安全・品質・リスク・コストを開発初期から並行検討し、後工程での手戻りを抑える

識別：IPT＝**人・チーム**、IPPD＝**開発の進め方・統合的アプローチ**。IPPDはIPTを組織的な実践手段として用い得る。

24. ソフトウェア開発手法・モデルの比較（CISSP学習用）

以下の「向く案件」は典型例であり絶対条件ではない。セキュリティはどのモデルでもSDLC全工程に組み込む。

手法	特徴・基本ステップ	向いている開発	セキュリティ上の注意／CISSPの着眼点
構造化プログラミング／構造化開発	順次・選択・反復など明確な制御構造でロジックを分解。一般的な進行：要件→機能分解→設計→実装→テスト	処理手順が明確な業務・バッチ処理	条件分岐・境界値・例外処理・入力検証を確認。構造化は開発規律でありセキュリティ保証ではない
オブジェクト指向開発（OOP）	クラス・オブジェクトを用い、カプセル化・継承・多態性等を活用。設計→実装→単体・統合試験	複雑な業務ドメイン、拡張・保守の多いアプリ	アクセス修飾子や不変条件、継承・ポリモーフィズムによる予期せぬ挙動、オブジェクトの安全なシリアライズに注意
ランタイムシステム	実行時にプログラムの実行を支援する基盤（VM、言語ランタイムなど）。**開発方法論ではない**	Java/.NETなどの実行環境	ランタイム自体の脆弱性、パッチ、実行権限、サンドボックス、依存ライブラリの管理

手法	特徴・基本ステップ	向いている開発	セキュリティ上の注意／CISSPの着眼点
反復型開発（Iterative）	**計画→設計→実装→評価→フィードバック→再反復**を繰り返す	要件が変わる、探索・改善が必要な製品	各反復でセキュリティ要件・回帰テスト・変更影響評価を繰り返す。反復＝無統制ではない
RAD（Rapid Application Development）	短期反復・プロトタイプ・利用者参加・ツール活用。代表的フェーズ：**要求計画→ユーザー設計→構築→切替**	迅速な画面・業務アプリ試作、利用者から頻繁にフィードバックを得られる案件	短納期で認証・認可・入力検証・文書化・十分なテストを省かない。高安全性システムでは慎重に適用
JAD（Joint Application Development／Joint Application Design）	利用者・業務部門・開発者等の共同ワークショップで要件や設計を合意する**協働技法**	関係者が多く要件の合意形成が難しい案件	セキュリティ・プライバシー担当を参加させ、アクセス制御や監査の要件を合意・文書化。**JAD自体は独立したSDLCモデルではない**
探索的モデル／探索的開発	実験・試作→評価→仮説修正→再試作。要件や解決方法を発見しながら進める	新技術検証、実現可能性が不確定な案件	PoC用の弱い設定やダミーコードを本番流用しない。試作と本番のセキュリティ基準を区別
CASE（Computer-Aided Software Engineering）	設計・コード生成・ドキュメント化・検査などをツールで支援。**モデルそのものよりツール群**	大規模開発の一貫性・効率化	生成物もレビューし、CASEツールのアクセス権・供給元・更新・成果物を保護
コンポーネントベース開発	既製・再利用可能部品を組み合わせて構築。選択→適合性評価→統合→テスト	共通機能が多いシステム、短期構築	サプライチェーン・依存性・脆弱性・ライセンス・部品の信頼境界。SCA/SBOMが有効
再利用モデル	承認済み資産・コードの探索→評価→調整→統合→検証	類似機能が繰り返し出る複数システム	**再利用される脆弱性も広範囲に波及**。コードの出所・安全性・版管理・再試験が必要
スパイラルモデル	**目標・代替案の決定→リスク分析／低減→開発・検証→次サイクルの計画**を周回。リスク駆動型	大規模・高リスク・仕様不確実なシステム	**各周回でリスクを評価**する点がCISSPの識別ポイント。コストと管理負担は大きい
プロトタイピング	要求把握→試作品→利用者評価→改良（必要なら本番設計）	UIや要求の確認、利用者要件が曖昧な案件	試作版の認証・ログ・例外処理が未完成なら本番投入しない。実データを安易に使用しない
MPM（Modified Prototype Model：修正プロトタイプモデル）	教材固有の名称・工程差があり得る。概念的には試作を評価し修正を反復し、確定版に反映する	初期要求の不確かさを試作で解消する案件	試作品から製品へ移行する際にセキュリティ要件の欠落を点検。**教材に示された正式な工程・定義は原文照合推奨**

手法	特徴・基本ステップ	向いている開発	セキュリティ上の注意／CISSPの着眼点
クリーンルーム開発（Cleanroom）	数学的・形式的仕様→正しさを重視した設計・検証→統計的使用テストなど。欠陥の予防重視	高信頼・高保証を目指すソフトウェア	**欠陥の予防と形式的検証**。通常の「開発者が大量のデバッグを繰り返す」方式とは重点が異なる
XP（Extreme Programming）	短い反復、ペアプログラミング、TDD、継続的インテグレーション、リファクタリング、頻繁なリリース	小〜中規模の変化の速い開発、密な顧客連携	TDDに**セキュリティテスト**を追加し、レビュー・脅威モデリング・依存性診断を省略しない
アジャイル開発	短期反復で動くソフトウェアを提供し、フィードバックにより優先度を変える。例：バックログ→反復計画→実装・試験→レビュー→振り返り	要件変動が大きく価値を小分けに届けたい製品	**セキュリティ要件もバックログに**、Definition of Doneにセキュリティ受入基準を含める
DevOps	開発と運用の協働・自動化・継続的デリバリー。計画→コード→ビルド→テスト→リリース→運用→監視→改善	Webサービスなど頻繁なデプロイや運用フィードバックを重視する開発	CI/CDの権限・秘密情報・署名・承認・監視・変更管理が重要。**DevOps＝セキュリティ統合とは限らない**
DevSecOps	DevOpsのパイプラインと文化にセキュリティを早期から統合。計画時の脅威分析→実装時の静的検査→依存性検査→動的試験→配備統制→実行時監視	頻繁な変更でも継続的にセキュリティ確認が必要なサービス	**Shift Leftと継続的検証**。SAST/DAST/SCA・秘密情報スキャン・IaC検査・ポリシーゲート・監視を自動化。最終リスク受容の権限は組織側

24.1 混同しやすい用語を分ける

- **SDLC**＝開発から運用・廃止までのライフサイクル。**ウォーターフォール／反復型／スパイラル**＝進め方のモデル。
- **RAD**＝短期・利用者参加・プロトタイプ重視。**JAD**＝利用者と開発者が要件・設計を共同で詰める技法。JADはRADで活用されることがある。
- **アジャイル**＝反復・適応の価値観と開発アプローチ。**XP**＝代表的な具体的実践を持つアジャイル手法。
- **DevOps**＝開発・運用の協働と継続的提供。**DevSecOps**＝そこへセキュリティを横断的・継続的に統合。
- **構造化プログラミング／OOP**＝コードの構成・設計思想。**CASE**＝開発支援ツール。**ランタイム**＝実行基盤。これらを同一階層の「開発工程モデル」と誤認しない。

24.2 試験直前：キーワードから選ぶ

問題文のキーワード	優先して連想するもの
リスクを各周回で繰り返し特定・軽減	**スパイラル**
短期間でプロトタイプと利用者フィードバック	**RAD／プロトタイピング**

問題文のキーワード	優先して連想するもの
利害関係者が共同ワークショップで要件合意	**JAD**
ペアプログラミング、TDD、リファクタリング	**XP**
変更に適応、短期反復、継続的な価値提供	**アジャイル**
開発・運用・自動デプロイ	**DevOps**
CI/CDでセキュリティ検査・ポリシーを自動化	**DevSecOps**
構成要素の再利用、外部部品の安全性	**コンポーネントベース／再利用モデル**
形式的な正しさ、欠陥予防、統計的品質試験	**クリーンルーム**
異分野の専門家が一体となって作業	**IPT**
製品と開発・テストプロセスを統合的に設計	**IPPD**

全モデル共通のCISSP原則：手法を選ぶ前に業務目標・安全性要件・リスク許容度を確認し、各工程でセキュリティ統制・テスト・変更管理を実施する。手法の名称だけで安全性は保証されない。

8.1 追加：ライフサイクル手法の選択と成熟度モデル

25. ソフトウェアライフサイクル手法の選択

評価基準	検討事項	代表的な選択例
要件の確実性	要件が確定しているか／変化しやすいか	変更が少ない→ウォーターフォール、変化が多い→反復型・アジャイル
技術的・事業上のリスク	失敗時の影響と不確実性	高リスク・不確実→スパイラル（各周回でリスク分析）
納期・利用者の参加	短期試作や密な利用者フィードバックが可能か	RAD・プロトタイピング・JAD
配備・変更の頻度	継続的に変更・リリースするか	DevOps、セキュリティの継続統合ならDevSecOps
コンプライアンス・安全性	証跡・独立検証・安全要求が厳格か	ライフサイクル手法を問わず要件追跡、承認、独立検証を組み込む

CISSP判断： 開発方式の名前ではなく、事業目的、リスク許容度、規制、変更可能性、チーム能力に合わせて選ぶ。

26. SW-CMM（Software Capability Maturity Model：ソフトウェア能力成熟度モデル）

目的： 組織のソフトウェア開発プロセスの成熟度を評価し、改善段階を示すモデル。個別プログラムのセキュリティ検査結果を直接示すものではない。

レベル	英語	日本語	状態と覚え方

レベル	英語	日本語	状態と覚え方
1	Initial	初期	属人的、場当たり的。成果は個人の力量に依存
2	Repeatable	反復可能	過去の成功した管理手順を繰り返せる。基本的なプロジェクト管理
3	Defined	定義された	組織共通の標準プロセスが定義・文書化され、教育・運用される
4	Managed	管理された	プロセスと品質を**定量的**に測定し、統計的・数値的に管理する
5	Optimizing	最適化	測定結果から継続的にプロセスを改善し、欠陥予防・革新を進める

覚える順番： 属人化 → 繰り返せる → 組織で標準化 → 数値で管理 → 継続改善。

混同注意： 元のSW-CMMではレベル2が **Repeatable**、レベル4が **Managed**。CMMIの成熟度表記では通常レベル2が **Managed**、レベル4が **Quantitatively Managed** となる。教材がSW-CMMを問う場合は教材の名称で答える。

27. OWASP SAMM（Software Assurance Maturity Model）

目的： ソフトウェア開発・運用に関わる**セキュリティ実践の成熟度**を測り、組織の状況とリスクに合わせて改善する。特定の開発手法には依存しない。

ビジネス機能	主な評価対象	主なプラクティス（整理用）
Governance（ガバナンス）	方針・推進体制・教育	Strategy & Metrics / Policy & Compliance / Education & Guidance
Design（設計）	セキュリティ要件・脅威・設計	Threat Assessment / Security Requirements / Security Architecture
Implementation（実装）	セキュアな構築・配備・欠陥修正	Secure Build / Secure Deployment / Defect Management
Verification（検証）	設計評価・テスト	Architecture Assessment / Requirements-driven Testing / Security Testing
Operations（運用）	インシデント・環境・運用	Incident Management / Environment Management / Operational Management

SAMM v2系の構造： 5つのビジネス機能、合計15のセキュリティプラクティス。各プラクティスについて成熟度レベル1〜3に向けた活動を評価する（未実施をレベル0として扱うこともある）。

改善の基本手順： 現状評価 → 目標成熟度の設定 → ギャップ分析 → 優先度付けした改善計画 → 実施・再評価。

CISSPでの比較： SW-CMMは**ソフトウェア開発プロセス全般の成熟度**、SAMMは**ソフトウェアセキュリティ実践の成熟度**。成熟度を一律に最大化するのではなくリスク・コスト・事業目的に応じて目標を設定する。

8.2 ソフトウェア開発エコシステムのセキュリティコントロールを特定し、適用する

---

1. 学習目的（今回読み上げた範囲）

教材読み上げの要旨

- ソフトウェアの**開発・配備・サポート**を管理することがセキュリティに与える影響を理解する。
- 成熟したプログラミング環境とツールセットがもたらすセキュリティ上の利点を説明する。
- **コードリポジトリとライブラリの利点・リスク**を説明する。
- **IDE（統合開発環境）**の価値を説明する。
- 開発サイクル短縮とコード再利用が拡大する一方、第三者コンポーネントを取り込む際のリスクが増える。

補足：登場する用語の位置づけ

用語	役割	利点	セキュリティ上のリスク・統制
ツールセット	コンパイラ、ビルド、テスト、解析など開発作業を支援する道具の集合	再現性と標準化、自動検査	ツール汚染、旧版放置、設定不備。承認済みツール、更新、権限分離
IDE（Integrated Development Environment）	エディタ、デバッガ、ビルド、補完、検査などを統合	コーディングミスを早期検出、効率向上	悪性拡張機能、秘密情報漏えい、脆弱なプラグイン。拡張機能の制限と更新
コードリポジトリ	ソースコードと変更履歴を保管・共同管理（例：Gitのホスティング）	変更追跡、レビュー、復元、協働	不正改変、認証情報流出、権限濫用。MFA、最小権限、保護ブランチ、レビュー、監査
パッケージリポジトリ	再利用するライブラリやパッケージの配布元	迅速な依存関係導入と更新	偽装パッケージ、依存性混乱攻撃、改ざん。信頼元制限、名称・版・整合性検証
ライブラリ	再利用可能な実装部品や関数群	開発短縮、既知の機能を標準化	既知の脆弱性、悪意あるコード、保守停止。導入前評価・継続監視・更新

重要な区別： Git等の**ソースコードリポジトリ**と、npm・PyPI等の**パッケージリポジトリ**は目的が異なる。どちらもソフトウェアサプライチェーンの保護対象。

2. 標準ライブラリ・各種ライブラリ・ソフトウェア再利用の階層

教材読み上げの要旨

教材は、開発時の選定に役立つ大まかなライブラリの分類として、以下を挙げている。これは**常に安全度が上から下へ機械的に並ぶランキングではない**。

種類（教材の順）	役割・例（補足）	代表的なメリット	注意点・推奨コントロール
① OS・ハードウェアライブラリ	OS API、ドライバ周辺のSDKなど	OS・機器の機能を標準手段で利用	特権境界、ドライバ署名、OS/SDKの更新、サポート期限

種類（教材の順）	役割・例（補足）	代表的なメリット	注意点・推奨コントロール
② プログラミング言語ライブラリ	標準ライブラリ、公式ランタイム提供機能	共通の実装・検証・互換性	言語・ランタイムのサポートと脆弱性、危険なAPIの誤用
③ 開発フレームワーク	Webフレームワーク、ORM等	開発構造の統一、共通機能	初期設定、認証・認可の使い方、依存性の連鎖、パッチ適用
④ 社内／プロジェクト／顧客／製品部門固有ライブラリ	社内共通認証モジュールなど	社内規約への適合、重複削減	所有者・保守責任・変更管理・レビュー・内部脆弱性の横展開
⑤ 信頼できる情報源が公開するサードパーティOSS	実績のある公開ライブラリ	開発期間短縮、コミュニティの知見	公開元・真正性、脆弱性、ライセンス、メンテナの状況を継続確認
⑥ 実績や提供組織の性質が未検証のサードパーティOSS	出所・管理者・保守実績が不明な部品	便利な機能を迅速に利用できる場合	**高い慎重さ**。出所、完全性、悪性コード、脆弱性、継続保守を確認し、承認まで利用を制限

「信頼できるライブラリ」の意味（補足）

有名・公式・ダウンロード数が多いというだけで**安全性は保証されない**。署名やハッシュが一致することは**配布物の真正性・完全性の確認**を助けるが、コードに脆弱性がないことは保証しない。OSSでも商用でも組織が採用リスクと保守責任を持つ。

3. 再利用前に確認する欠陥・悪意あるコード

教材読み上げの要旨

再利用候補は、**古典的なコーディングエラー、トロイの木馬、論理爆弾**などを確認すべき対象として挙げられている。

脅威・欠陥	意味	例	主な対策
古典的なコーディングエラー	入力・境界値・例外処理などの実装欠陥	不正な入力によるバッファ境界外アクセス、認可漏れ	SAST、コードレビュー、単体・結合テスト、ファジング
トロイの木馬（Trojan Horse）	正常なソフトウェアに見せかけて不正な動作を含む	便利なライブラリを装い認証情報を外部送信	配布元の検証、依存性検査、コード審査、実行時挙動の監視
論理爆弾（Logic Bomb）	特定条件や日時などで悪意ある処理を発火	特定日にデータを削除する隠し処理	コードレビュー、変更履歴検査、職務分離、最小権限、監査

重要： これらの検査を実施したからといって「再利用して絶対に安全」とはならない。**使用目的への適合性、ライセンス、依存先、更新可能性、残余リスク**も評価する。

4. ライブラリ採用・再利用のセキュリティ管理：実務手順（補足）

1. **要件・利用目的の確認**：何の機能に必要で、どの情報・権限に触れるかを特定。
2. **供給元評価**：公式配布先、メンテナンス実績、作者・運営体制、サポート、ライセンスを調査。
3. **真正性・完全性確認**：パッケージ名、バージョン、署名／ハッシュ、取得経路を確認。名前の似た別パッケージにも注意。
4. **コード・依存性評価**：SCA（Software Composition Analysis）で直接・間接依存を把握。既知脆弱性、危険な実装、悪意ある振る舞いを検査。
5. **承認と安全な統合**：社内ポリシーに従う。特権を最小化し、検証環境でテストする。
6. **継続的な監視・更新**：SBOM（Software Bill of Materials：ソフトウェア部品表）や依存性台帳で利用先を追跡し、新たな脆弱性・保守終了時に改修・置換する。

コード再利用の重要なトレードオフ（補足）

- 良い再利用は同じ実装の作り直しを減らすが、**脆弱な共通部品を多数の製品で再利用すると影響も広がる。**
- SCAは主に**組み込まれた部品と既知の脆弱性などの依存性リスク**を把握するための仕組み。SASTは自社コード等の静的解析、DASTは動作中のアプリケーションに対する動的検査。目的は異なる。
- SBOMは構成要素を可視化する**一覧**であり、SBOMの作成だけでは脆弱性を修正できない。

5. CISSP試験の識別ポイント

問題文での手掛かり	判断の軸
開発生産性を上げ、人的なミスを早期発見したい	標準化されたツールセット・IDE、静的検査、レビュー
ソースの改変や履歴を追跡したい	リポジトリのアクセス制御、ブランチ保護、変更管理、監査
OSS・外部部品の脆弱性が心配	サプライチェーン評価、SCA、SBOM、更新管理
知名度の高いライブラリだから安全か	**知名度は保証ではない。**供給元・版・設定・依存性・脆弱性を確認
再利用により開発を高速化したい	品質・保守性の利点と**脆弱性の共通化・波及**というリスクを比較
正規プログラムを装う悪性コード	**トロイの木馬**
特定条件で不正動作が発火	**論理爆弾**
ライブラリの導入を承認する責任	開発者任せにせず、組織のリスク・変更・供給元管理に従う

Domain 8.2 続編：ツールセットの評価から開発エコシステムの保護まで【2026-10-27追記】

6. 共通プログラミング言語ライブラリ：5つの利点

教材で示された利点	意味・具体例	注意するリスク（補足）

教材で示された利点	意味・具体例	注意するリスク（補足）
信頼性の向上	広く使われ十分テストされた共通機能を再利用し、独自実装でのミスを減らす	有名でも未修正の欠陥や設定不備はあり得る
プロセスリスクの低減	同一機能を開発者ごとに実装する不統一を減らし、標準的な実装を採用	ライブラリの採用・更新・廃止の管理が必要
専門家の効果的な活用	暗号など高度な機能は専門家が整備した実装を利用できる	暗号ライブラリを使っても鍵管理・使い方の設計は残る
標準への準拠	標準プロトコルやAPI仕様への適合を容易にする	ライブラリの利用だけで法令・規格への組織全体の準拠は保証されない
開発の加速	認証・通信・入出力などの共通処理の再実装を省く	依存性管理不足、古い版の放置、サプライチェーン攻撃

CISSPの判断： 「車輪の再発明」を減らすメリットと、**共通ライブラリの不具合が広範囲に波及する**リスクを一体で評価する。再利用する前に出所、脆弱性、ライセンス、依存先、更新方針を確認する。

7. プログラミングツールとツールセット

7.1 IDE（統合開発環境）

IDE（Integrated Development Environment）はエディタ、ビルド、デバッグ、テスト、コード補完、バージョン管理連携などを一体化した開発環境。コード品質の向上、標準化、開発効率の改善を支援する。

- **利点：** 型エラーや構文の早期検知、Lint、静的解析の統合、単体テスト実行、レビュー支援。
- **リスク（補足）：** IDE拡張機能・プラグインの不正コード、資格情報の流出、ローカル開発環境のマルウェア、過剰権限。
- **対策（補足）：** 承認済み拡張の利用、更新管理、開発用資格情報の保護、署名・配布元の検証、ソースコードへの最小権限。

7.2 SAST・DAST・IAST・RASP：まとめて比較

技術	実行状態・観測位置	何を見つける／守るか	主な長所	主な制約
SAST（Static Application Security Testing）	原則コード・バイナリ等を**実行せず**解析	危険な関数、入力処理、データフロー上の欠陥	開発早期に実行でき、コード位置を示せる場合がある	設定や実運用の挙動が見えにくい。言語・ビルド情報依存や誤検知がある
DAST（Dynamic Application Security Testing）	**動作中**のアプリに外部からリクエスト	実際の応答に現れる注入・設定・認証関連の一部問題	ソースコード不要の場合が多く、外部からの挙動を評価	テストケースで到達しない処理を見逃し、原因位置の特定が難しい場合がある

技術	実行状態・観測位置	何を見つける／守るか	主な長所	主な制約
IAST（Interactive Application Security Testing）	**動作中**のアプリをエージェント等で内部観測	リクエストと内部データフローを関連づけた欠陥	DAST相当の動作検証とコード実行状況を結び付けやすい	エージェント対応やテスト範囲に依存し、性能負荷等に注意
RASP（Runtime Application Self-Protection）	**実行時にアプリ内部・近傍**で監視・防御	不審な入力や実行動作の遮断・緩和	攻撃発生時の追加防御に活用できる	誤遮断、パフォーマンス、迂回・未対応攻撃の可能性。脆弱性修正の代替ではない

覚え方： SAST＝**静かにコードを見る**／DAST＝**動かして外から試す**／IAST＝**動かして内側も観測**／RASP＝**動作中に守る**。

リファクタリング（Refactoring）＝外部から観察できる振る舞いを原則変えずに、内部の構造や可読性・保守性を改善すること。**セキュリティ検査そのものではない**が、複雑さや重複を減らし、レビューと保守を容易にする。変更後の回帰テスト・セキュリティ確認が必要。

7.3 自動解析の弱点・制約（教材の重要項目）

制約	意味	CISSP上の補完策
ビジネスロジック・認証／認可の欠陥	文脈依存のアクセス制御の不備は自動ツールだけでは判定しにくい	脅威モデリング、権限マトリクス、手動レビュー、認可テスト
暗号の不安全な利用	暗号化されていても弱い設定・誤った鍵管理・不適切な利用は残る	暗号設計レビュー、設定監査、鍵管理プロセスの評価
誤検知（False Positive）	実際には問題でない箇所を警告する場合がある	優先順位付け、検証、例外承認の記録。偽陰性（見逃し）にも注意
構成の不備はソース外にある	本番環境の権限・クラウド設定・環境変数・秘密情報の扱いがコードにないことがある	構成管理、IaC検査、CSPM、環境設定レビュー、実環境試験
コード・依存性・ビルド情報の不足	ライブラリ、コンパイルオプション、生成コードが不足すると正しい解析が難しい	完全な依存関係・ビルド情報の管理、SCA/SBOM、可能な範囲での再現可能ビルド

結論： 解析ツールを導入するだけで安全を保証できない。**SAST／DAST／IAST、レビュー、テスト、構成監査、依存性管理を組み合わせる**。

8. 開発・テスト用サンドボックスとJavaアプレット

Sandbox（サンドボックス）＝実行プロセスのファイル、ネットワーク、メモリ、権限等を制限し、ほかの領域への影響を抑える分離環境。開発時の試験、不審なプログラムの解析、ブラウザやモバイルアプリの実行制限などに利用できる。

場面	目的	注意点

場面	目的	注意点
開発・テスト用サンドボックス	不安定なコード、検査対象を本番資産から分離	実データや本番秘密情報を持ち込まない。外部通信やホストとの共有を制限
ブラウザ内の実行制御	ダウンロードしたコードが端末に無制限アクセスしないよう制約	サンドボックス突破・ブラウザ脆弱性は残り得る
Javaアプレット（**歴史的な技術**）	過去には署名や実行許可等に応じ、サンドボックス制限の対象となった	現代の主要ブラウザではJavaプラグイン／アプレットは原則サポートされない。署名済み＝安全ではない

教材の「アプレットはブロックされる」「サンドボックスアプレット／トークンアプレット」について： 読み上げられた名称の後半は原語の確認が必要。試験上は、**信頼できない実行コードを制限するサンドボックス**の目的と、**署名・許可による権限制御の歴史的な考え方**を区別する。現代の推奨実装としてJavaアプレットを扱わない。

9. ソフトウェア開発エコシステムにおけるセキュリティコントロール

9.1 変更管理とマルウェア対策の両方が必要

- **変更管理：** 変更要求→影響・リスク評価→承認→実装・テスト→展開→記録・監査。勝手なコード変更、ライブラリ置換、設定変更を防ぐ。
- **悪意あるソフトウェアへの対策：** 端末・ビルドサーバーの防御、署名／ハッシュ、依存性チェック、最小権限、供給元の監視、異常なビルド成果物の検知。
- **CISSPの狙い：** 「正規の変更として紛れた悪意あるコード」と「単純なミスによる欠陥」の両方を視野に入れる。

9.2 環境のコントロールと分離

環境	目的	代表的な対策
開発（Development）	開発・デバッグ、試行錯誤	開発者権限の管理、模擬データ、コード・秘密情報の保護
品質保証／テスト（QA/Test）	セキュリティ・機能・回帰・受入テスト	本番近似の設定、独立テスト、テストデータの匿名化／合成
本番（Production）	実利用者向けサービス	厳格な変更承認、最小権限、監視、可用性・復旧策

最重要： 開発者が本番データを自由に利用したり、開発環境から本番へ承認なく直接変更したりしない。**職務分離（Separation of Duties）、アクセス制御、環境分離**が基本。環境ごとに鍵・資格情報・ネットワークを適切に分離する。

9.3 IDE・プログラム実行・ランタイム環境

- **IDE保護：** プラグイン管理、拡張の供給元検証、ローカル端末管理、リポジトリ認可、開発用シークレットの保護。
- **ビルド・実行環境保護：** CI/CD実行者、ビルドエージェント、コンパイラ、パッケージ管理、依存ライブラリを保護。署名済み成果物・監査証跡の活用。

- **ランタイム保護：** OS／ミドルウェア更新、権限分離、アプリの隔離、ログ・監視、セキュア構成、脆弱性管理。

9.4 コードリポジトリのセキュリティ

守る対象	代表的な統制
機密性：ソース、設計情報、秘密	最小権限、MFA、秘密情報スキャン、暗号化、アクセス監査
完全性：コミット・ブランチ・成果物	保護ブランチ、Pull Requestレビュー、署名、変更承認、改ざん検知
可用性：開発資産の継続利用	バックアップ、復元テスト、リポジトリの継続管理、災害対策
追跡性：誰が何を変更したか	バージョン履歴、監査ログ、Issue／変更要求との紐付け

例： ライブラリのアップデートやCI/CDワークフロー変更はコード以外の変更も含め、レビュー・承認する。リポジトリ管理者・ビルドシステムの侵害は供給先に波及し得る。

10. OS・ハードウェアによる安全な配備：TCB／参照モニター／セキュリティカーネル

10.1 概念の関係

TCB（Trusted Computing Base：信頼できるコンピューティング基盤）＝システムのセキュリティポリシーを強制するために依存するハードウェア、ファームウェア、ソフトウェアの構成要素全体。ここに欠陥があると、システムの保護に深刻な影響が及び得る。

Reference Monitor（参照モニター）＝主体（ユーザー・プロセス等）が客体（ファイル・資源等）へアクセスする際、セキュリティポリシーの適用を仲介する抽象的な概念。

Security Kernel（セキュリティカーネル）＝参照モニター概念を実現する、TCBのうちアクセス制御・保護強制を担う中核部分。OSカーネル全体と必ずしも一致しない。

用語	一言	例・意味
TCB	**信頼を置く保護機構の全体**	CPUの特権モード、メモリ保護、OSのアクセス制御、保護に関わるファームウェア等
参照モニター	**アクセスを必ず審査する概念**	ユーザーがファイルへアクセスするたびに権限確認する仕組みの論理モデル
セキュリティカーネル	**参照モニターを実装する中核**	カーネルの権限制御やメモリ保護に関わる部分

10.2 参照モニター／セキュリティカーネルの3要件（重要）

要件	意味	失敗例
完全仲介（Complete Mediation）	**すべてのアクセス**を仲介し、チェックを迂回させない	特定APIから権限検査なしにファイルを開ける

要件	意味	失敗例
耐改ざん性（Tamperproof / Isolation）	アクセス制御機構自身を非特権主体が改変できないよう**保護・隔離**する	一般ユーザーがアクセス判定コードを書き換えられる
検証可能性（Verifiability）	十分小さく単純で、正しく動くことを分析・テスト・評価できる	制御機構が巨大で複雑すぎて検証困難

読み上げの注意：「完全隔離、Isolation、それから検証可能性」と挙げられた部分は、一般に**完全仲介／耐改ざん性（隔離）／検証可能性**の3つとして整理する。**完全仲介（Complete Mediation）と隔離（Isolation）は別概念**なので同義語にしない。

セキュリティと設計の関係（補足）： TCBはできる限り小さく、単純で、検証可能に設計する。OSの特権分離、Secure Boot、TPM、メモリ保護、署名検証などは状況に応じて保護を支援するが、どれか1つがTCB全体の安全を保証するわけではない。

11. より安全なソフトウェアのための暗号化アプローチ

11.1 教材で挙げられた組織ポリシー

ポリシー項目	具体的な実施内容と目的
デジタル証明書、暗号スイート、鍵の管理	許可されたアルゴリズム・強度・プロトコル・証明書の発行／更新／失効・鍵のライフサイクルを定義する
パスワード／トークン管理とIAMの統合	認証情報の保管・ローテーション、MFA、最小権限、ライフサイクル管理を統合する
HSMの活用	重要な暗号鍵の生成・保管・使用を耐タンパー性を備える専用機器／サービス内で保護する（証明書そのものは一般に公開情報であり、主な保護対象は**秘密鍵**）
ITシステム調達と暗号ポリシーの整合	調達前に対応暗号方式、鍵管理、証明書運用、更新能力、要件への適合を評価する
Citizen Developer／BYOD／BYOx／Shadow ITへの対応	IT部門外の開発、私物端末、未承認サービスも対象としてデータ・秘密情報・アクセスを保護する。例外を無管理にしない
開発・テスト等の特殊環境への暗号サポート	開発用証明書・テスト用秘密鍵の適切な管理。本番秘密鍵の流用防止、環境ごとの資格情報分離

用語の補足： 音声の「自然デベロッパー」は、文脈から**Citizen Developer（市民開発者）**を指す可能性が高い。**BYOD＝Bring Your Own Device**、**BYOx／BYOE＝Bring Your Own Everything**（用法に幅あり）。Shadow IT＝正式な管理・承認の外で使用されるIT資産やサービス。

11.2 CISSP試験上の注意点

- **暗号化しただけで安全とは限らない。** 鍵管理、暗号方式、証明書の検証、構成管理、廃止・失効の手続きを含めて評価する。
- **HSMは秘密鍵保護の仕組み。**HSMを導入するだけで認可、証明書の発行審査、アプリの実装不備が解消されるわけではない。

- **開発・テスト環境で本番秘密鍵や機密データを安易に使わない。** セキュリティ統制は本番だけの仕事ではない。
- **持込み・非公式開発は例外として放置しない。** 業務目的を評価したうえで、管理対象化・安全な代替手段・教育・技術統制を選択する。

12. 今回の直前確認表

問題文のキーワード	優先して想起すること
再利用で品質・速度・標準化を改善	共通ライブラリの利点＋供給元／版／脆弱性の管理
開発初期・コードを実行せず解析	**SAST**
実行中アプリを外部から検査	**DAST**
実行時に内部観測しながら検査	**IAST**
実行中の攻撃を検知・ブロック	**RASP**
内部構造を改善し振る舞いを維持	**リファクタリング**
未信頼コードの実行を隔離	**サンドボックス**
開発・QA・本番を分ける	**環境分離・職務分離**
セキュリティ方針の実施を信頼する要素全体	**TCB**
全アクセスを仲介する抽象的仕組み	**参照モニター**
参照モニターを実装する中核部分	**セキュリティカーネル**
完全仲介・耐改ざん・検証可能	**参照モニターの3要件**
暗号の秘密鍵の保護	**HSM**
開発・テスト用の鍵	**本番と分離して管理**

8.2 続編｜DBMS、構成管理、CI/CD、セキュリティコントロールの粒度【2026-10-27追記】

13. データベース管理システム（DBMS）のセキュリティコントロール

13.1 DBMSの目的と保護対象

教材読み上げ： DBMS（Database Management System）のコントロール、ロックコントロール、ACID特性、ビュー／GRANT・REVOKE／オブジェクト指向データ／メタデータ／データ汚染のコントロール、OLTPのセキュリティを扱う。

学習補足： DBMSはデータの定義、格納、検索、更新、トランザクション処理、アクセス制御、バックアップ、復旧などを管理する基盤。CISSPでは「CIAを守りつつ**許可されたトランザクションだけが、矛盾なく、必要な期間維持される**」という考え方を押さえる。

統制領域	代表的な対策	目的・試験での論点
認証・認可	IAM連携、DBロール、最小権限、ビュー、行レベルセキュリティ（RLS）	誰がどの表・行・列を読める／変更できるか
トランザクション	ACID、適切な分離レベル、ロック、コミット／ロールバック	更新の整合性、競合処理
機密性	保存時・通信時暗号化、鍵管理、マスキング、トークナイゼーション	個人・機密データの露出抑止
完全性	主キー・外部キー・CHECK制約、入力検証、監査ログ	不正・不整合な値を防止
可用性	バックアップと復元試験、レプリケーション、フェイルオーバー	障害からの復旧、RPO/RTOを満たす
変更・監視	DDL変更承認、パッチ、設定ベースライン、監査ログ	構成ドリフト、特権濫用、不正更新の検知

注意： アプリがSQLインジェクションに脆弱な場合、DB側の権限制御だけですべてを防げるわけではない。アプリ側の**パラメータ化クエリ**、認可、入力検証、DBアカウント最小権限を多層で実施する。

13.2 ロックコントロール（Locking Control）

目的： 複数の利用者／処理が同じデータを同時に更新・参照するとき、整合性の破壊や不適切な競合を防ぐこと。

用語・方式	挙動	具体例・注意
共有ロック（Shared / Read Lock）	複数の読み取りを許し、競合する書き込みを制限	同じ在庫データを複数人が参照
排他ロック（Exclusive / Write Lock）	更新対象への競合アクセスを制限	残高更新時に同時更新の衝突を防止
楽観的同時実行制御（Optimistic Concurrency Control）	更新の衝突が少ないと仮定し、更新時にバージョン等を確認	バージョン不一致なら再試行／競合処理
悲観的同時実行制御（Pessimistic）	先にロックし競合アクセスを防ぐ	競合が多い処理。待ち時間・デッドロックに注意
MVCC（Multi-Version Concurrency Control）	複数のデータ版を利用し、読み取りと書き込みの競合を緩和	スナップショット読み取り。分離保証はDBと設定に依存
デッドロック（Deadlock）	処理AとBが互いのロック解除を待って進めない	DBによる検知・片方のロールバック等で解消

学習上の区別： ロックは**同時アクセスの調停手段**。ACIDは**トランザクションに求める性質**。同じ概念ではない。DBMSの分離レベルやMVCCの実装によってロックの振る舞いは異なる。

13.3 ACID特性（最重要）

教材読み上げ： Atomicity、Consistency、Isolation、Durabilityの4つ。音声の「ACIDテスト」は、一般には **ACID特性（ACID properties）**と呼ばれる。

文字	英語	日本語	意味	例
A	**Atomicity**	**原子性**	一連の処理は**全部成功するか、全部取り消す**（All or Nothing）	口座Aから減額後、口座Bへの加算に失敗したら、減額もロールバック
C	**Consistency**	**一貫性／整合性**	トランザクション前後で定義された**制約・整合性規則**を守る	残高のCHECK制約、外部キー整合性を維持
I	**Isolation**	**分離性／隔離性**	同時トランザクションの干渉を所定の分離レベルに従って制御する	同じ在庫を同時購入しても不整合な更新を防ぐ
D	**Durability**	**永続性**	**コミットが成功した変更は障害後も失われない**よう保証する	コミット済み送金記録をログ・永続ストレージから復旧できる

重要な訂正： Isolationは音声では「独立性」と読まれたが、DBMSの一般的な日本語訳は**分離性／隔離性**。厳密には並列実行が常に完全逐次実行と同じになるわけではなく、**設定された分離レベル**により許容する現象が変わる。

ACIDと混同しやすい現象

現象	内容	主な関連項目
ダーティリード（Dirty Read）	未コミットの変更を別処理が読んでしまう	Isolation
非反復読み取り（Non-repeatable Read）	同じ行を二度読むと別処理のコミットにより結果が変わる	Isolation
ファントムリード（Phantom Read）	同じ条件で再検索すると対象行の集合が変わる	Isolation
ロストアップデート（Lost Update）	競合した更新の一方が他方を上書きして失われる	Isolation／同時実行制御
ロールバック（Rollback）	処理失敗時にトランザクションを取り消す	Atomicity
コミット（Commit）	変更を確定する	Atomicity／Durability

実務イメージ： 銀行振込ではA（片側だけ減算しない）、C（残高等の制約を維持）、I（同時振込を適切に処理）、D（振込確定後に電源断でも結果が残る）。**ACIDは暗号化やユーザー認証を意味しない。**

13.4 その他のDBMSアクセス制御

制御	意味	使用例	CISSPでの注意
ビューベース（View-Based Access Control）	元テーブルの一部列・行を仮想的な表示で提供し、公開範囲を制限	人事担当には氏名と部署のみ、給与詳細は表示しないビュー	ビュー経由だけのアクセスに制限する。元表への直接権限があれば迂回できる

制御	意味	使用例	CISSPでの注意
GRANT／REVOKE	DBオブジェクトに対する権限を**付与／取り消し**	`GRANT SELECT ON ...`、`REVOKE INSERT ON ...`	最小権限、ロール単位、権限棚卸し。REVOKEの挙動はDBMSの権限継承等で差がある
オブジェクト指向データの保護	オブジェクトのデータと操作（メソッド）へのアクセスを制御	`Customer`オブジェクトの個人情報フィールドやメソッドに認可を適用	カプセル化は保護に寄与するが、**自動的に認可を保証しない**。継承・関連オブジェクト経由の漏出にも注意
メタデータコントロール	スキーマ、データ辞書、テーブル定義、索引、接続情報などの閲覧・変更を制御	システムカタログ参照やDDL実行を管理者に限定	データ本文が非公開でも、構造・機密列名から情報が推測される。スキーマ変更も監査
データ汚染のコントロール	不正・破損・信頼できないデータが後続処理に伝播することを抑える	入力検証、データ品質ルール、取り込みデータ隔離、出所追跡	「Data Contamination」は教材の語。一般に**データ品質・完全性・出所管理**として考える。攻撃コード混入も別途点検

データ汚染（Data Contamination）の具体例

取引CSVの金額欄に誤値や不正な文字列が混入 → 取り込み処理 → 売上集計・レポート・データレイクへ伝播。対策は「入口での形式・範囲・業務整合性検証」「信頼度による隔離」「エラーの報告と修正」「監査・トレーサビリティ」。**Garbage In, Garbage Out（GIGO）**との関係も整理する。データ汚染はウイルス感染と同義ではない。

13.5 OLTP（Online Transaction Processing）のセキュリティ

OLTP＝オンラインで発生する多数の比較的小さな業務トランザクションを迅速・正確に処理する仕組み。例：銀行口座、決済、EC注文、航空券予約、在庫更新。

主なリスク	望ましい統制
同時更新で在庫・残高が不一致	ACID、分離レベル、ロック／MVCC、競合検出
不正な取引・二重決済	強固な認証・認可、業務ルール検証、冪等性キー、異常検知
注入攻撃・不正入力	パラメータ化SQL、型・範囲・意味の入力検証、DB最小権限
障害で取引履歴が失われる	トランザクションログ（WAL等）、バックアップ、フェイルオーバー、復元試験
不正な閲覧・改ざん	TLS・保存時暗号化、DB権限、監査証跡、職務分離
高負荷・可用性低下	容量計画、レート制限、冗長化、監視、障害試験

OLTPとOLAPの対比（補足）：OLTPは**日々の注文・支払など取引の更新**、OLAP（Online Analytical Processing）は**集計・分析・意思決定支援**。OLAPも保護が必要だが、OLTPでは特に高頻度の整合性ある更新・可用性が重要。

14. ソフトウェアセキュリティの側面としての構成管理（CM）

14.1 構成管理の4つの主要活動

教材読み上げ： 変更の「特定、コントロール、アカウンティング、監査」。構成コントロールは、変更の承認・配信などに関連する活動であり、自動化・オーケストレーションとも関係する。

活動	英語	内容	例
構成識別	Configuration Identification	管理対象（CI）、バージョン、関係、ベースラインを**特定**する	ソースコード、ライブラリ版、設定、IaC、ビルド成果物の一覧
構成制御	Configuration Control	ベースラインへの変更を評価・承認し、許可された内容を反映する	RFC・レビュー・承認・マージ・リリース制御
構成状況記録	Configuration Status Accounting	いつ・誰が・何を・どの版に変更したか**記録・報告**する	Git履歴、変更チケット、リリース台帳、SBOM
構成監査	Configuration Auditing	実物・変更履歴・文書・承認・ベースラインが整合するか**照合**する	本番構成が承認済み設定に一致するか検査

注意： 「構成コントロールは最後の3つのフェーズを処理する」という読み上げ表現は、教材での「Configuration Control」の広義の使い方かもしれない。一般的なCMでは上記4活動を区別し、**構成制御は特に変更を許可・実装する活動**として扱う。混同せず、教材固有の説明として記録する。

14.2 CM計画（Configuration Management Plan）に含めるもの

項目	何を決めるか	セキュリティ上の理由
対象範囲と構成品目	アプリ、OS、DB、ライブラリ、IaC、開発ツール等	管理漏れの防止
役割と権限	申請・評価・承認・変更・検証・監査の担当	職務分離、最小権限
ベースラインと版管理	正式版、タグ、リリース、依存関係	既知の安全な構成を再現
変更プロセス	RFC、影響分析、テスト、承認、ロールバック	未承認変更を防ぎ、障害に備える
構成状況の記録	誰・いつ・何を・なぜ変更したか	追跡可能性・説明責任
監査と例外処理	定期照合、ドリフト監視、緊急変更の事後承認	実態と承認状態の乖離を是正
保管・復旧	バックアップ、成果物保管、完全性検証、再現可能ビルド	改ざん・消失・リリース事故への備え

構成管理 vs 変更管理 vs リリース管理

- **構成管理（CM）**：何がどの状態・版で存在するかを識別・制御・記録・監査する。
- **変更管理（Change Management / Change Enablement）**：変更のリスク、影響、承認、手順を管理する。
- **リリース／デプロイ管理**：承認済み変更群を製品としてまとめ、対象環境へ安全に提供する。自動化しても**承認・記録・検証は不要にならない**。

14.3 情報保護管理（Information Protection Management）

教材読み上げ： CMに関連する「情報保護管理」の重要性が言及された。正式な個別工程や用語定義は読み上げられていないため、以下は**理解のための補足**。

構成情報・ソースコード・設計図・秘密情報・DB定義・ビルド手順・製品成果物を情報資産として扱い、分類、アクセス制御、暗号化、バックアップ、改ざん検知、監査、保持期間・安全な削除を管理する。特に構成ファイルに埋め込まれた**秘密鍵・APIキー・DBパスワード**は漏えいしやすいので、専用シークレット管理基盤で保護する。

15. CI/CD（継続的インテグレーション／継続的デリバリー）

15.1 定義と流れ

- **CI（Continuous Integration）**：開発者の変更を共有ブランチ等に頻繁に統合し、**自動ビルド・テスト・解析**で問題を早期発見する。
- **Continuous Delivery（継続的デリバリー）**：検証済みの変更を**必要なときに本番リリースできる状態**へ継続的に整える。手動承認・配備判断が残る場合がある。
- **Continuous Deployment（継続的デプロイメント）**：定義されたゲートを満たした変更を、追加の人手承認なしに本番へ**自動デプロイ**する方式。**DeliveryとDeploymentは同義ではない**。

```
変更要求・コード作成
  → バージョン管理／プルリクエスト・レビュー
  → CI：ビルド → 単体／統合テスト → SAST・SCA・秘密検査
  → 成果物の署名・保管／SBOM・証跡
  → CD：ステージング配備 → DAST・受入／構成検査
  → リスクに応じた承認（Delivery）または自動本番配備（Deployment）
  → 本番監視 → フィードバック／変更管理
```

15.2 CI/CDで特に保護すべき場所

対象	主なリスク	推奨統制
ソースリポジトリ	不正コミット、ブランチ乗っ取り	MFA、コードオーナー承認、保護ブランチ、署名検証
ビルド実行環境	悪性ジョブ、ランナーの乗っ取り	分離、短命の実行環境、最小権限、パッチ
CI/CDシークレット	トークン・鍵の漏えい	専用保管、短期資格情報、環境分離、監査
依存ライブラリ	供給網攻撃、既知脆弱性	SCA、SBOM、許可ソース、固定版、更新管理
成果物・コンテナ	配布途中での差し替え	署名、ダイジェスト検証、アクセス制御
本番デプロイ権限	未承認の本番変更	承認ゲート、職務分離、変更記録、ロールバック

試験の判断： CI/CDを導入するとセキュリティも自動的に確保される、は誤り。DevSecOpsでは**セキュリティチェックをパイプラインに組み込み**、例外と残余リスクを権限者が管理する。

16. セキュリティコントロールの粒度（Granularity）

16.1 教材読み上げ内容の整理

教材の音声「緩度」は文脈から**粒度（Granularity）を指すと考えられる。アクセス制御で指定できる範囲・対象・アクションの細かさ**のこと。粒度が粗いと、業務上不要な権限を与えざるを得なくなりやすい。

粒度	設定例	問題・効果
粗い（Coarse-Grained）	「人事部の人は人事DBのすべてのテーブルを読める」	必要以上の個人情報閲覧を許してしまう
細かい（Fine-Grained）	「給与担当は担当社員の給与列を参照、更新は特定ワークフローのみ」	**最小権限・Need-to-Know**を実現しやすい

粒度に関わる制御軸： 主体（ユーザー／サービス／ロール／属性）、対象（システム／DB／表／行／列／レコード）、操作（参照／作成／更新／削除／承認）、条件（時間／場所／端末状態／データ分類／取引金額）。RBACやABAC、DBの行・列レベル制御は細粒度の認可に役立つ。

16.2 CISSP上の注意

- **粒度が細かい＝必ず安全**ではない。ルールが膨大・複雑になると誤設定、権限競合、レビュー漏れが起こる。**運用可能な粒度**を選ぶ。
- **最小権限（Least Privilege）**＝仕事に必要な最小の権限。**Need-to-Know**＝情報を知る必要性。関連するが同じ概念ではない。
- アクセスコントロールは**プログラムとユーザーの双方**に適用する。サービスアカウントや自動処理に過大なDB権限を持たせない。
- 最初に**保護すべきデータ／業務機能とアクセス要件を把握**し、その要件を実装できる適切な粒度を選ぶ。粒度を細分化すること自体が目的ではない。

17. 今回のCISSP直前識別チェック

問題文で目立つ言葉	最優先で想起すること
全部成功か全部取り消し	**ACID：Atomicity（原子性）**
制約・整合性を保つ	**ACID：Consistency（一貫性）**
同時トランザクションの干渉	**ACID：Isolation（分離性）**
コミット済み結果が電源断後も残る	**ACID：Durability（永続性）**
同時更新の衝突	**ロック、MVCC、適切な分離レベル**
表の一部だけ公開	**ビュー／列・行のアクセス制御**
DB権限の付与・取り消し	**GRANT／REVOKE**
スキーマ・データ辞書の保護	**メタデータコントロール**
不正な入力が集計まで広がる	**データ汚染のコントロール／GIGO**
高頻度な決済・在庫更新	**OLTP、ACID、監査、可用性**

問題文で目立つ言葉	最優先で想起すること
構成を識別・制御・記録・監査	**構成管理の4活動**
変更の影響分析・承認・ロールバック	**変更管理／構成制御**
コード変更ごとにビルド・テスト	**CI**
いつでも本番リリース可能な状態	**Continuous Delivery**
条件を満たした変更が自動で本番へ	**Continuous Deployment**
権限設定が粗く、必要以上のアクセス権	**粒度（Granularity）不足**
行・列・操作・条件ごとに権限設定	**Fine-Grained Access Control**

8.3 ソフトウェアセキュリティの有効性を評価する

8.3.1 目的と全体像

教材読み上げの要旨

- 従来の**構成管理（Configuration Management：CM）と自動構成ツール**を比較する。
- システム開発環境で、適切な**セキュリティテスト**を計画・実施する方法を理解する。
- **ソフトウェアアシュアランス（Software Assurance：SwA）**によって得られる信頼と、費用・期間・人的資源のトレードオフを考える。
- セキュリティの状態を継続的に測定し、業務・ミッション要件やリスク管理へフィードバックする。

補足：CISSPの大きな流れ

事業要件・リスク把握 → CMで対象と変更を統制 → 検証・評価 → 結果の記録・報告 → 権限者の判断 → 継続的改善。

「テストを実施した」だけでなく、**何を・どの基準で・誰が・どの権限で判断したか**が重要。ソフトウェアの安全性は単一のテストで保証されない。

8.3.2 リスクとビジネスニーズの比較：4段階

教材読み上げで示された流れ

順序	フェーズ	目的	システム開発・セキュリティの例
1	**発見（Discover）**	事業目的、資産、利用者、脅威、制約、法規制、ギャップを把握	顧客情報を扱うサービスで何を守るべきか調査する
2	**評価（Assess / Evaluate）**	ビジネス影響・リスク・実現可能性・対策の費用対効果を分析	流出リスクと対策候補、残余リスク、対応の優先順位を評価
3	**同意（Agree）**	関係者間で優先事項、受入基準、責任・予算・リスク対応方針を合意	経営・システムオーナー・開発・セキュリティ担当が要件と許容条件に合意

順序	フェーズ	目的	システム開発・セキュリティの例
4	**文書化（Document）**	合意内容を方針、仕様、基準、計画に落とし込み追跡可能にする	セキュリティ要件・構成ベースライン・テスト基準・例外判断を記録

注意：この4段階は今回の教材で示された「ニーズ特定の整理」。NIST RMFの正式ステップ名と同一視しない。

8.3.3 従来型CMと自動構成ツール／構成コントロール（CC）

教材読み上げの要旨

- **「測定できなければ管理できない」**という原則に基づき、セキュリティ、安全性、レジリエンスに必要なCM/CCを設計する。
- 管理対象の構成要素と**管理粒度**を決定し、正式な**構成項目（Configuration Item：CI）**を定義する。
- 変更の識別・承認・実装状況・ログ・監査によって、構成ベースラインと実態の差異を管理する。

CMとCCの区別（補足）

項目	意味	具体的な活動
CM（Configuration Management）	システム構成をライフサイクル全体で把握・管理する包括的プロセス	構成識別、変更統制、構成状況記録、構成監査
CC（Configuration Control）	CMの一部として、構成変更を評価・承認・統制する活動	RFCの影響分析、承認、リリース条件、ロールバック、追跡
CI（Configuration Item）	識別・管理・変更履歴の対象となる構成要素	ソースコード、ライブラリ、設定ファイル、DBスキーマ、コンテナイメージ、IaC定義等
Baseline	正式に承認された比較基準の構成	リリース版、設定標準、ハッシュ／バージョン情報

CIという略称の衝突：Domain 8.2の **CI＝Continuous Integration（継続的インテグレーション）** とは文脈で区別する。**構成管理のCI＝Configuration Item。**

従来型と自動化の比較（補足）

観点	従来の手作業中心CM	自動化されたCM／構成ツール
変更把握	チケット、台帳、手動レビューに依存しやすい	ソース管理・構成収集・IaC差分検出・自動監査を利用
利点	非定型・例外事象の人間による判断を反映しやすい	一貫性、再現性、証跡、迅速な検証とドリフト検出
リスク	更新漏れ、属人化、遅延、記録の不整合	ツールや自動化権限の侵害、誤設定の大量展開、パイプライン障害
適切な統制	承認と職務分離、台帳・ログ・監査	最小権限、コードレビュー、変更承認、署名・完全性検証、監査ログ、ロールバック

CISSPの考え方： 自動化はCM統制を支援するが、**責任者の承認・リスク受容・変更管理を不要にするものではない。**

変更監査・ロギング（補足）

対象CIごとに、**誰が／いつ／何を／なぜ／承認者は誰か／実施結果はどうか** を記録する。変更申請、影響分析、承認、実施、テスト、検証、復旧記録を関連づける。構成の変更履歴は改ざん防止・適切な保存期間・アクセス制御・時刻同期も必要。監査は「承認済みのベースラインと実際の構成が一致するか」を確かめる。

8.3.4 ソフトウェアアシュアランスとセキュリティアセスメント

教材読み上げの要旨

- ビジネス・ミッションニーズの変化により、リスクアセスメントやリスク管理の方針が変わり得る。
- 継続的セキュリティアセスメントで発見した課題は、**情報分類、アクセス認可基準、セキュリティポリシー**等の見直しにつながる。
- 組織のある部門が行うセキュリティ・リスク低減策は、他部門の判断や業務手順とも関連する。アシュアランスを独立した単発活動にしない。

ソフトウェアアシュアランス（SwA）の定義

教材では、次の状態について**合理的な信頼**を得られることが重視される。

期待される性質	具体的な意味	例・検証の着眼点
意図した機能を果たす	定義された業務・セキュリティ機能が仕様や利用目的を満たす	認証・決済・帳票・監査ログ等が期待どおり動く
意図しない／許可されない機能を実行しない	不正な副作用、隠し機能、権限外操作を抑制する	バックドア、過剰権限、誤った管理機能を検査
悪用可能な脆弱性の排除・低減	要件・設計・コード・構成・運用中に弱点を作り込まず検出・是正する	脅威モデリング、レビュー、脆弱性検査、ペネトレーションテスト
ライフサイクル全体で保護される	設計・コード・成果物・データ・配備・更新の完全性等を守る	リポジトリ権限制御、変更管理、署名、監査

厳密な注意（学習補足）：「脆弱性がない」を**数学的に完全保証できる**という意味ではない。アシュアランスは、定めた要求と評価証拠に基づく**信頼の程度**を高めるもの。テストで何も発見されなくても、欠陥がゼロとは証明できない。

SwAは「プロセス」であり「結果」でもある

側面	意味	実務例
プロセス	計画的・体系的にエビデンスを収集し、評価し、要求への適合とリスクを判断する一連の活動	要件レビュー→設計・コード分析→テスト→欠陥是正→再評価→報告

側面	意味	実務例
結果	オーナー、利用者、利害関係者がソフトウェアの安全性・信頼性について持ち得る**合理的な確信**	評価報告書、残余リスク、保証レベル、リリース判断根拠

セキュリティアセスメントは、特定の統制・実装・運用状態が適切か検査・評価する手段。SwAは、これら複数の証拠を含めて**ソフトウェアの安全性への信頼を積み上げる広い考え方**。

アシュアランスとコスト・期間・資源のバランス（補足）

高リスクな医療・決済・基幹業務では、独立した検証、広範なテスト、形式的レビュー等への投資が正当化されやすい。一方、すべてのシステムに同じ深さのテストを強制することは費用・期間を圧迫する。**データ分類、脅威、事業影響、法規制、リスク許容度に応じてアシュアランスの水準を選ぶ**。

8.3.5 セキュリティ評価に使う4つの正式なアプローチ

教材で列挙された4種類を整理する。各アプローチは目的・適用範囲が異なり、同列の「具体的テスト技法」ではない。

教材の4種類	役割・焦点	CISSPでの区別
認証・認定（Certification & Accreditation：C&A）	セキュリティ統制を評価し、組織として運用の可否やリスク受容を判断する（旧来の制度・呼称を含む）	技術的評価と、**権限者による正式な運用判断**を分ける
リスク管理フレームワーク（RMF）	リスクをライフサイクルに沿って管理する枠組み	システム境界・分類・統制選択・実装・評価・認可・監視などを連携させる
ソフトウェアプロセスCMM	開発・管理プロセスの成熟度を段階的に評価・改善する	「組織の開発プロセスの成熟度」であり、**個別ソフトの脆弱性不存在証明ではない**
ソフトウェア品質保証／ソフトウェアアシュアランス	プロセスと製品の品質・安全性に関する証拠を集め、評価・改善する	SQA＝品質要件全般。SwAはセキュリティ・安全性等に対する信頼も含める文脈がある

補足： 8.1で扱ったSW-CMM・CMMI・OWASP SAMMは名称・評価対象・成熟度体系が異なる。**CMM＝プロセス成熟度、SAMM＝セキュア開発活動の成熟度**と区別する。

8.3.6 ソフトウェアアシュアランスポリシー

教材の要旨

組織はSwAを場当たり的なテストに任せず、**文書化され、適切に説明・周知され、関係者に理解されたポリシーとプロセス**を整備する必要がある。

ポリシーに含めたい要素（補足）

- 対象システム・開発体制・委託先・製品のリスク分類と適用範囲。
- 必要なセキュリティ・品質要件、受入基準、評価の頻度・手段。
- セキュアコーディング規約、レビュー、試験データ、証跡、独立性、職務分離。
- 脆弱性の報告・修復期限・例外処理・再試験・変更統制。
- 経営層／オーナー／開発／QA／セキュリティの責任分担とリスク受容権限。
- 継続的な教育・監査・見直しと改善。

8.3.7 認証・認定・認可（Certification / Accreditation / Authorization）の整理

教材の「認証と認可、認定」は日本語で混同しやすい。**文脈を確認することが重要**。

用語	この章での意味	代表的な主体／成果	混同注意
Certification（認証・評価）	安全性・統制の実装や適合を技術的に評価する、旧来のC&A文脈など	評価担当／評価報告・適合判断	ログイン時の本人認証とは別
Accreditation（認定）	評価結果を踏まえ、システム運用と残余リスクを正式に認める旧来の用語	権限者／運用に関する正式判断	現代のNIST RMFでは主にAuthorizationという表現を使う
Authorization（認可）	指定された権限者が、残余リスクを認識し運用を承認すること（Security Authorization）	Authorizing Official（AO）／認可判断	ユーザーへのアクセス権の付与（access authorization）とは区別
Authentication（本人認証）	利用者・サービスの身元の確認	IAM、認証システム／本人性確認	今回のC&Aの議論では主題ではない

試験キーワード：「評価結果を踏まえてシステムの残余リスクを受容し、運用を許可するのは誰か」→**権限を有する認可担当者／AO**。評価員・開発者が勝手にリスク受容を決めない。

8.3.8 テスト・検証・セキュリティアセスメント：基本的な位置づけ

教材のテスト定義

テストは、**制御された条件**（初期状態・指定入力・期待結果）でシステムを動作させ、実際の結果が必要な結果と一致するかを確認する活動。

- 修復、保守、機能追加はOS、ネットワーク、アプリ、UI、基盤構成など多様なレイヤーで起こる。
- 変更によって新機能の受入基準が満たされるか**受け入れテスト**で確認し、既存機能へ副作用が生じていないか**回帰テスト**で確認する。
- ワークフロー、環境構成、運用時の条件を再現して測定する。単発テストでは不十分な場合、継続テストや運用評価を組み合わせる。

主要なテストの比較（CISSP向け補足）

種類	何を確かめるか	いつ・誰が実施するか	典型例

種類	何を確かめるか	いつ・誰が実施するか	典型例
受け入れテスト（Acceptance Test）	**受け入れ基準／業務要件を満たすか**。発注者・顧客が受け入れる判断を支援	リリース／引渡しの前、顧客・オーナー側が関与	新しい振込上限機能が業務・セキュリティ要件を満たす
回帰テスト（Regression Test）	**変更が既存機能を壊していないか**	修正・更新のたびに、自動・手動で再実施	振込画面修正後も残高計算や権限制御に不具合がない
単体テスト（Unit Test）	個々の関数・部品が期待どおりか	実装時、開発者中心	価格計算関数や認可判定関数を確認
統合テスト（Integration Test）	構成要素・外部サービス間のインターフェースが正しいか	部品統合後	APIとID基盤・DBの連携で権限漏れがない
システムテスト（System Test）	システム全体が要件に適合するか	結合後・引渡し前	全機能と性能、可用性、監査ログ等を確認
セキュリティアセスメント／コントロールテスト	**統制の設計・実装・運用上の有効性**	リスクと変更に応じて継続的に	アクセス制御テスト、ログ完全性、暗号設定、セグメント確認
倫理的ペネトレーションテスト	合意された範囲で悪用可能性と影響を実証・評価	事前承認・スコープ・停止条件の下で	認可回避や横展開が可能か安全に検証
運用テスト・評価（OT&E）	実利用条件に近い形で機能・安全性・運用適合性を確認	利用側の関与、運用環境に近い条件	障害時運用、実トラフィック想定、ユーザー手順

比較の核心：受け入れテストは「**新しい要求を満たすか**」、回帰テストは「**既存が壊れていないか**」。一つの変更に両方が必要になることがある。

運用時アセスメントと要件変更の関係

教材では、実運用における使用方法・条件が初期要件を超えて拡大する可能性に触れている。**現状の利用実態・新しい脅威・変更後の構成**を評価し、結果をリスク管理やポリシー見直しへつなげる。初期に合格したシステムも、利用・環境・脅威が変われば再評価が必要。

8.3.9 コントロールのテストと評価：テストシナリオ・データの選定

教材の要旨

セキュリティ評価では、**適切なシナリオ、評価担当者、テストデータ**を用意する。データは正常範囲内の代表値だけでなく、許容範囲の端、範囲外、想定外、例外条件を含める必要がある。

テストデータ設計（補足）

データ区分	意味	例：送金金額の上限が100万円の場合
正常値・代表値	一般的な有効入力を試す	1万円、30万円

データ区分	意味	例：送金金額の上限が100万円の場合
境界値（Boundary Value）	上限・下限およびその直前・直後を試す	99万9,999円、100万円、100万1円
範囲外／異常値	仕様上認めない入力を試す	100万1円超、負の金額、極端に大きな値
欠損・形式不正	空欄、文字・型・長さの不一致を試す	金額なし、abc、想定外の桁数
意味的不正（Semantic）	型・範囲は正しくても業務ルールと矛盾する	非許可口座への振込、限度額の異なる利用者の操作
権限・セッション異常	認証済み／未認証、権限差、セッション期限等を試す	他人の口座IDを指定、権限変更直後、期限切れトークン
負荷・並行／障害条件	同時処理、競合、障害復旧を試す	二重送金リクエスト、DB切断、タイムアウト

テストの管理：本番の機密データの利用は原則避け、可能なら合成・匿名化／適切にマスキングしたデータを用いる。テスト実施者の権限、環境隔離、ログ保護、結果の文書化、データ廃棄も統制対象。

受け入れ基準・評価結果をどう扱うか（補足）

1. **要件・統制目的**を明確にする（例：権限外の振込操作を拒否）。
2. **評価基準・想定結果**を定義する（拒否応答、監査ログ記録、金額不変）。
3. 正常・境界・異常・権限・例外の**テストケースとデータ**を準備する。
4. 実行・観測して、**結果・証拠・欠陥**を記録する。
5. 欠陥をリスクに応じて是正し、**再試験・回帰テスト**を実施する。
6. 残余リスクと証跡を報告し、**適切な権限者**が受け入れ可否を判断する。

8.3.10 CISSP直前暗記：設問キーワード → 判断

出題の表現	まず連想する判断軸
ニーズの特定の4段階	**発見 → 評価 → 同意 → 文書化**
変更を管理する対象単位	**構成項目（CI＝Configuration Item）**とその粒度
構成の実態が承認済み標準と違う	**ベースライン比較／構成監査／ドリフト検知**
手作業の変更管理を自動化する	**一貫性・追跡性を向上。ただし統制と承認は維持**
自動化で設定ミスを大量に広げる	**パイプライン権限、事前検証、段階的展開、ロールバック**
ソフトウェアアシュアランス	**計画的な評価プロセス＋得られる合理的な信頼**
ソフトのプロセス成熟度の評価	**SW-CMM／CMMI**（個別製品の安全性そのものとは別）
セキュア開発活動の成熟度	**OWASP SAMM**（8.1）
リスクを把握して本番運用を正式に許可	**AOによるSecurity Authorization**
顧客が受け入れ基準を確認	**Acceptance Test**
改修で既存機能が壊れていないか	**Regression Test**

出題の表現	まず連想する判断軸
仕様どおりの実装か	**Verification**
利用目的に合っているか	**Validation**
予期しない悪用可能性を評価	**セキュリティアセスメント／許可を得たペネトレーションテスト**
上限値・上限の直前・直後を試す	**境界値分析**
形式上正しいが業務ルールに反する入力	**意味的な入力検証**
テストで問題なし＝絶対安全か	**No：範囲・手法・証拠に基づく合理的な確信のみ**

8.3 全体の結論

セキュリティ要求を測定可能にし、構成と変更を追跡し、適切なシナリオとテストで有効性を評価し、結果をリスク管理と経営上の判断につなげること。 どのテストや成熟度モデルもそれ単独で安全性を証明するものではなく、必要なアシュアランス水準をビジネスのリスクと費用に応じて選ぶ。

8.4 取得したソフトウェアのセキュリティへの影響を評価する

8.4.1 学習目的と本章の全体像

教材読み上げの要旨

- **サードパーティー・ベンダー、COTS（Commercial Off-The-Shelf）、コモディティシステム、孤立したシステム要素**について、セキュリティアセスメントを支援する方法と成果物を特定する。
- **合併・買収（M&A）**に伴うソフトウェア、データベース、システムのセキュリティ評価上の問題を説明する。
- 内製か外部調達かにかかわらず、**資産オーナー／調達組織のリスク管理責任は残る。**外部サプライヤーの成熟度や宣伝文句だけで、導入の安全性を保証したことにはならない。

調達・取得方式を並べて理解する（学習補足）

方式・対象	何を取得／利用するか	主な利点	重視するセキュリティ確認
サードパーティー開発	ベンダーに依頼した専用ソフト	自社にない技能、短期導入の可能性	開発プロセス、セキュアSDLC、契約要件、検収、脆弱性対応、知財・保守
COTS	既製の商用ソフト	開発の省力化、標準化、サポート	サプライヤー評価、設定、パッチ、サポート期限、製品依存関係
OSS（オープンソース）	ライセンスに従いソースを利用できるソフト	検証可能性・再利用・柔軟性	メンテナーの信頼、脆弱性、依存性、SBOM、ライセンス、更新・署名
旧式／孤立システム	既存だが保守が難しい要素	業務継続に不可欠な場合がある	補完的統制、隔離、監視、代替・撤退計画、リスク受容

方式・対象	何を取得／利用するか	主な利点	重視するセキュリティ確認
MSP・MSSP	運用／セキュリティ運用の外部サービス	専門家・運用体制の利用	**責任分担、SLA、監査権、データ管理、退出戦略**
SaaS・PaaS・IaaS	クラウド経由のソフト／基盤サービス	俊敏性、拡張性、従量課金等	共有責任モデル、ID、構成、暗号、ログ、可用性、契約

8.4.2 サードパーティー開発によるソフトウェア取得：CMMだけでは足りない

教材読み上げの要旨

- ソフトウェア開発組織は、開発プロセスの能力評価・改善に**CMM（Capability Maturity Model）**等を利用することがある。
- ただし、一般的な成熟度モデルの高い評価だけでは、**ソフトウェアの安全性・セキュリティ・アシュアランスを十分に実践していることを必ずしも意味しない。**
- 調達する組織のセキュリティ専門家は、**サプライヤーの開発能力にセキュリティアシュアランスが統合されているか**、そして組織のリスク低減要件を満たすかを確認しなければならない。

実務上の評価項目（補足）

評価領域	サプライヤーに確認する証拠例	評価の意図
開発プロセス	SDLC、CMM/CMMI関連評価、品質管理記録	工程が標準化・改善されているか
セキュア開発	セキュリティ要件、脅威モデル、コードレビュー、SAST/DAST/SCA、脆弱性修正履歴	**品質成熟度と別にセキュリティ能力を実証**できるか
供給網・ビルド	依存部品の一覧（SBOM）、署名、ビルドの完全性、権限分離	改ざん・脆弱な部品の混入を防げるか
運用・インシデント	パッチ提供期限、重大脆弱性の通知、IR連絡先・報告期限	導入後のリスクを管理できるか
ガバナンス・契約	監査権、下請け管理、データ処理条件、終了時の引継ぎ	責任・証拠・是正を契約で支えられるか

CISSP頻出のひっかけ：CMM成熟度が高い＝製品が安全、とは限らない。 開発能力の証拠と、セキュア開発・テスト・運用の客観的な証拠を別々に確認する。CMM、CMMI、OWASP SAMMの違いは8.1も参照。

8.4.3 取得時のソフトウェアアシュアランス：段階的なアプローチ

教材読み上げで示された全体の流れ

企画 → 契約 → 監視・承認・配備 → 継続的な使用とサポート → 廃棄／置換

> 教材は「企画」「契約」「監視・承認・配備」「継続的な使用とサポート」を中心に説明している。廃棄・廃止措置は継続的な使用とサポートの中で明示されている。監視、承認、配備の区切り方は教科書の表によって異なり得る。

A. 企画フェーズ（Planning）

まず事業・ミッション上のニーズを特定する。 単なる機能調達ではなく、情報資産、必要な保護、長期運用まで考える。

教材で説明された活動	実施の意味・成果物	セキュリティ専門家の評価点
ニーズと業務プロセスの明確化	当初ニーズを詳細化し、既存業務プロセスへの変更を特定	セキュリティ・プライバシー・法規制要件を最初から記載
初期規模・スコープ・概算見積もり	必要な作業・資源・期間の大まかな見積もり	テスト、評価、改善、保守に必要なコストを含む
リスクアセスメント	**開発・導入しないリスク**と、**開発・受け入れ・運用するリスク**を比較	不作為リスクと調達・供給網リスクを両方評価
実装代替案の比較	コスト、スケジュール、リスク、意思決定要件を比較	内製・外注・COTS・クラウド・現状維持も比較対象となり得る
ROI（投資収益率）・ハードル率の検討	投資対効果を推定し、組織の投資採択基準と比較	残余リスク、安全性、規制遵守を単純なROIに還元しない
SOWに反映する要件と成果物の特定	要件、情報資産文書、テストデータ、アシュアランス記録、構成管理・変更記録、検証可能な成果物一覧	**成果物・証拠・受入条件・引渡形式**を明文化
取得計画とサプライヤー選別基準	調達リスク、候補者適格性・事前選別条件を定義	セキュアSDLCの証拠、組織の調達ポリシーとの一致
入札評価基準・評価計画	サプライヤー比較と意思決定の方法を策定	セキュリティを価格・納期と並ぶ明確な評価軸にする

用語：ROI＝Return on Investment（投資収益率）。**ハードル率**＝組織が投資案件に要求する最低収益率等の基準。**SOW**＝Statement of Work（作業範囲記述書／作業指示書）。要件・作業・納品物・責任分界・検収条件を規定する。

音声の「CMOおよび制御データ」は原文の略語が不鮮明。ここでは確認できる範囲の**構成管理（CM）・構成コントロール（CC）関係の証拠や記録**として整理し、CMO固有の正式名称は断定しない。

B. 契約フェーズ（Contracting）

実施事項	内容	CISSP上の注意
RFPの作成・提示	**Request for Proposal（提案依頼書）**。SOW、応札要領、提案評価、契約条件、事前資格などを記載	セキュリティ・保守・監査・報告・データ処理を契約前に要求
サプライヤー提案の評価	入札／提案内容を合意済み評価基準と照らして評価	実証されたセキュリティ能力と約束だけの宣言を区別
契約条件の調整・締結	条件変更、責任・成果物・受入・紛争・契約終了等を確定	セキュリティ義務と**検証する権利**を契約に明記

RFPとSOWの違い：RFPは候補企業から「何を、いくらで、どのように提供できるか」を募る文書。SOWは契約・発注対象の**具体的な業務・成果物・受入条件**を示す文書。両者は関連するが同義ではない。

C. 監視・承認・配備フェーズ（Monitoring / Acceptance / Deployment）

教材での**3つの主要活動**：

1. **契約作業スケジュールの設定・合意**：進捗報告、チェックポイント、成果物提出とレビューの時点を決める。
2. **変更管理・構成コントロール（CC）手順の実装**：要求変更、バージョン、設定、承認記録、変更影響評価を維持する。
3. **ソフトウェア成果物のレビューと承認**：機能・セキュリティ・品質・ドキュメント・受入基準に対する証拠を評価し、承認・配備する。

補足：独立したセキュリティテスト、脆弱性の残件リスト、ペネトレーションテスト結果、運用・保守手順、バックアウト計画、ライセンスとSBOM等は有益な証拠となる。**検収は「動いたからOK」ではなく、要求されたアシュアランスの証拠に基づく。**

D. 継続的な使用とサポートフェーズ（Continued Use / Support）

教材での**2つの主要活動**：

1. **リスク管理・アシュアランスケース管理・変更管理**：導入後も製品更新、脆弱性、公表されたCVE、運用状況に応じて証拠とリスク判断を更新する。
2. **持続性確保・廃棄／廃止措置**：長期保守、サポート終了への移行計画、安全なデータ消去、依存関係の整理、後継製品への置換を計画する。

Assurance Case（アシュアランスケース）：システムが特定のセキュリティ／安全性要求を満たすと主張する際、**主張（Claim）・議論／論拠（Argument）・根拠（Evidence）**を構造化して説明するもの。単なる「安全宣言」ではない。

取得ライフサイクルのまとめ

段階	中心となる問い	代表的なセキュリティ成果物
企画	何が必要で、何が危険か	リスク分析、要件、SOW案、調達戦略、評価基準
契約	供給者に何を義務付けるか	RFP、契約条項、SLA、セキュリティ・監査要件
監視・承認・配備	要求どおり安全に作られ納入されたか	進捗記録、変更履歴、テスト証拠、受入記録
継続使用・サポート	安全性を継続し、いつ終了するか	パッチ・リスク管理、アシュアランスケース、廃止／移行計画

8.4.4 OSS（オープンソースソフトウェア）とセキュリティアセスメント

定義・教材の趣旨

OSS（Open-Source Software）は、対応するオープンソースライセンスに基づいてソースコードの利用・調査・改変・再配布等が認められるソフトウェア。利用条件はライセンスごとに異なる。**OSS＝無償／無条件／無保証／セキュア、ではない。**

Security through Obscurity（隠蔽によるセキュリティ）：「内部構造やソースコードが秘密だから安全」という考えだけに依存する方式。教材では一般に有効なセキュリティ統制とはいえない趣旨が述べられている。秘密保持は防御の一部となり得るが、**設計の安全性・強固な認証・権限分離・検証の代わりにはならない。**

OSSの長所と注意点

観点	利点	注意・必要な対策
ソースコードの可視性	第三者がレビュー・監査可能	公開されていても誰かが十分に監査した保証はない
再利用・エコシステム	実装を短縮し標準技術を利用可能	脆弱性・依存ライブラリ・保守状態を評価する
変更・カスタマイズ	必要に応じた改変が可能	派生版のパッチ管理・検証・責任が増える
供給網	広範な知見と協働を活かせる	偽パッケージ、依存関係混同、メンテナー侵害、悪意ある更新に注意
ライセンス	利用条件が明確な場合が多い	コピーレフト等の条件、再配布義務、法務審査も必要

セキュリティアセスメントの流れ（補足）：取得元・署名／ハッシュ確認 → ライセンス確認 → 依存関係とSBOM把握 → 既知脆弱性・保守状況・レビュー → 検証環境でテスト → 本番採用判断 → 更新・脆弱性監視。スキャンは万能ではなく、**サプライヤー信頼性と運用統制が必要。**

8.4.5 孤立したソフトウェアとシステム（Orphaned Systems）／レガシーシステム

教材読み上げの要旨

元の製造元・開発者がなくなった、または**製品サポートや第三者のソフトウェア／ファームウェアサポートを受けられなくなった**システム要素が存在する。ソースコードや保守資料、設計文書がなくてもセキュリティ評価には価値があり、CVEなどから脆弱性を把握できる。ただし修正できないものもあるため、別の方法でリスク低減を検討する。

レガシーと孤立の違い（特に試験向け）

対象	定義	例	セキュリティ判断
レガシー（Legacy）	古い技術・設計・機能に依存する既存システム。**現在もサポートされる場合がある**	古い技術で構築した基幹DB、長期間稼働する業務アプリ	旧設計・保守制約を評価し、継続使用か近代化を検討
孤立（Orphaned）	**実効的な開発者／サポート主体がいない**ため更新・修正が難しい	ベンダー撤退済みの独自制御ソフト、保守契約不可のファームウェア	修正不能リスクを把握し補完的統制・代替・廃止を計画

重要：レガシー＝必ず孤立、孤立＝必ず古い、ではない。 両者は重なる場合もあるが判定基準が異なる。

修正不能な脆弱性に対する代替策（補足）

統制	具体策	制約・注意
分離・隔離	ネットワークセグメント分割、外部接続の遮断、ジャンプホスト	隔離だけでは内部不正・物理侵入を解決しない
補完的コントロール	FW許可リスト、仮想パッチ／IPS、アプリケーション許可リスト	**脆弱性そのものが直ったわけではない**
監視・最小権限	通信ログ、EDR適用可能性、アクセスログ、管理者権限制限	古いOSで新製品が動作しない場合もある
リスク処理・事業継続	業務依存・安全性・残余リスク評価、代替策の承認	現場の運用継続性と安全を無視しない
撤退計画	後継製品、移行、データ変換、EOL対応、廃棄	**期限・責任者・費用**を定めないと恒久化しやすい

CVE＝Common Vulnerabilities and Exposures。公開された脆弱性の識別子体系。**CVEは発見済み脆弱性の全てを網羅するわけではなく、掲載＝直ちに修正可能とも限らない**。資産・バージョン・実環境・悪用状況で優先度を評価する。

8.4.6 M&A（合併・買収）に伴うセキュリティアセスメント

教材で示された問題点

- 買収側は、対象組織の**既知・未知・状態不明のIT資産と情報資産を引き継ぐ**。
- 互いの**構成管理（CM）ベースライン、IT資産管理、情報資産管理**の定義・記録様式が不整合、欠落、価値不明であることがある。
- ソフトウェアライセンス、リビジョン、更新／パッチ適用レベルを含む**資産インベントリ**が不完全である。
- **リスクインベントリ、リスク低減戦略、プロセス、ポリシー**が両社で互換性を持たないか、未整備な場合がある。

M&Aの実務的な手順・統制（学習補足）

1. **デューデリジェンス**：買収前／可能な範囲で資産、データ分類、規制、サポート、インシデント、委託先、ライセンス、重大な脆弱性・技術的負債を評価する。
2. **統合前の暫定防護**：未知資産を含む環境をいきなり相互接続しない。信頼境界を定め、必要最小限の接続・ID連携にする。
3. **資産・構成の再発見**：インベントリ、CMDB／構成ベースライン、パッチ・更新状態、ネットワーク接続、権限、データフローを整理する。
4. **リスク・ポリシーの統合**：リスク評価基準、責任者、アクセス権、データ保管・越境、バックアップ、ログ・監査を整合させる。
5. **移行・改善と継続監視**：優先度を付けて是正し、インシデント対応・BCP・契約・ライセンスを統合する。

問題状況	避けたい短絡的判断	CISSP的な優先対応
買収先の資産一覧がない	既存セキュリティ基盤へ直ちに全面接続	**資産発見・分類、リスク評価、接続の制限**
CMベースラインが不明	すべての設定を即日統一	**影響分析、ベースライン確立、変更管理**

問題状況	避けたい短絡的判断	CISSP的な優先対応
ライセンスと保守の情報が不明	調達済み扱いで統合継続	**法的・契約上の権利とサポートを確認**
組織ごとに異なるID・アクセス権	一括で管理者権限を共通化	**最小権限、職務分離、権限棚卸し**
リスク評価基準が統一されていない	数値を単純合算して報告	共通評価尺度とリスク受容権限の整備

CISSPの原則：取得／統合後に何を防御すべきか不明なら、まず**資産・構成・データ・責任関係の可視化**。安全を保証しない状態での安易な統合は避ける。

8.4.7 コモディティシステム、MSP・MSSP・ERP

コモディティシステム（Commodity Systems）

組織専用ではなく、広く市場で提供される標準化された機器・ソフトウェア・サービスを組み合わせたシステムを想定する。入手性・費用・相互運用性に利点がある一方、**同じ既知の欠陥が多数の導入先に波及**し得る。製品の普及度や専門事業者への委託のみで安全性が保証されるわけではない。

用語	正式名称・概要	主な利用目的	セキュリティ上の注意
MSP	**Managed Service Provider**：ITインフラ・運用管理を請け負う事業者	監視、保守、バックアップ、ヘルプデスク等	特権アクセス、外部委託リスク、SLA、可用性、監査
MSSP	**Managed Security Service Provider**：セキュリティ運用・監視の外部提供者	SOC、SIEM監視、検知・対応支援、脆弱性管理等	対応権限、責任分界、報告・エスカレーション、ログと証拠の所有・保護
ERP	**Enterprise Resource Planning**：企業の経営資源・基幹業務を統合するシステム	会計、販売、在庫、人事、調達等を一体管理	職務分離（SoD）、重要データの機密性・完全性、承認ワークフロー、変更管理

MSPとMSSPの比較：MSPは主にITサービスの運用管理、MSSPは主にセキュリティサービスに重点。ただし提供範囲は重なり得る。**ERPはサービスプロバイダーではなく基幹業務システム／ソフトウェアの類型**である。

委託のCISSPポイント（補足）

- **SLA（Service Level Agreement）**：提供時間、応答、復旧、報告等のサービス水準。**責任分担のすべてをSLAだけで規定するわけではなく、契約・セキュリティ付属条件等も重要。**
- 外部ベンダーに業務を委託しても、**組織の説明責任と残余リスク管理はなくならない。**
- データの所在・所有権、アクセスログ、監査権、通知期限、下請け利用、BC/DR、契約終了時のデータ返却・安全な削除を確認する。

8.4.8 クラウドサービス：SaaS・PaaS・IaaSとCapEx／OpEx

サービスモデルの違い

モデル	正式名称	主な提供範囲	利用側が重点管理する例
SaaS	Software as a Service	完成したアプリケーションをサービス提供	**ユーザーID／アクセス権、テナント設定、データ分類と共有、利用監査**
PaaS	Platform as a Service	実行環境・ミドルウェア等の開発／運用プラットフォーム	**アプリコード、データ、ID、アプリ設定、依存部品**
IaaS	Infrastructure as a Service	仮想計算・ネットワーク・ストレージ等の基盤	**ゲストOS、アプリ、パッチ、ネットワーク設定、ID、データ**

大前提：クラウドの具体的な責任分担は**契約・構成・サービス内容**による。上表は典型例であり、SaaSだから利用企業にセキュリティ責任がないわけではない。

CapEx と OpEx

用語	意味	代表例	CISSP／経営判断
CapEx（Capital Expenditure）	**資本的支出**：取得した固定資産等への投資	自社サーバー購入、データセンター設備投資	初期投資が大きいことがある。減価償却等の会計上の扱いも考慮
OpEx（Operating Expenditure）	**営業費用／運用費用**：継続的な業務活動の支出	クラウド利用料、運用委託料、保守費用	費用が継続的・変動的になる場合。総所有コスト（TCO）と利用量を確認

注意：オンプレミスにもOpEx（電力・保守等）があり、クラウド契約にも前払い・長期契約等がある。したがって「オンプレ＝CapExのみ／クラウド＝OpExのみ」と断定しない。

クラウド・外部サービスの評価観点（補足）

- **共有責任モデル**：提供者／利用者がそれぞれどの層の安全性を担うか。
- **ID・権限管理**：MFA、フェデレーション、特権アクセス、サービスID。
- **データ保護**：暗号化、鍵の所有・管理、データ所在地、データの返却・削除。
- **構成・脆弱性管理**：責任範囲内のパッチ／設定、API設定、監査ログ。
- **可用性・継続性**：SLA、バックアップ・復元試験、事業継続、退出可能性・ベンダーロックイン。
- **契約・規制**：適用法、委託先・再委託先、監査、報告期限、規制遵守。

8.4.9 CISSP試験直前の判断表

問題文のキーワード	連想する概念・適切な第一歩
サプライヤーが高いCMM成熟度を主張	**ソフトウェアアシュアランスの客観的な証拠を別途確認**
ソフトウェア調達を開始する	事業要件・セキュリティ要件・リスク・代替案を定義
取得しない場合のリスク	**不作為リスク**も導入リスクと同時に評価
調達時の作業・成果物・検収を規定	**SOW**
候補業者に提案を求める	**RFP**

問題文のキーワード	連想する概念・適切な第一歩
導入後のアシュアランスを継続	リスク・変更・証拠・パッチ・保守期限を継続管理
主張と証拠で安全性を説明	**Assurance Case（Claim / Argument / Evidence）**
公開ソースなので安全か	**No**：コード監査可能性と実際の安全性は別
隠しておけば安全	**隠蔽だけに依存する設計は不適切**
古いがベンダー保守あり	**Legacy**：必ずしもOrphanedではない
開発元が存在しない／サポート不能	**Orphaned**。隔離・補完的統制・置換計画を検討
パッチ不可なCVE	影響評価・リスク受容権限・補完的統制・移行判断
買収先資産が見えない	**まず資産／構成の発見・分類・リスク分析**
ITインフラの外部管理	**MSP**
SOC監視等を外部委託	**MSSP**
会計・人事・販売等の統合業務基盤	**ERP**
SaaS利用で設定ミスや過剰権限	**利用企業にもID・データ・テナント設定の責任**
OSのパッチを利用者が管理する構成	典型的には **IaaSのゲストOS**
大きな初期設備投資	**CapEx**
毎月の従量利用・運用費用	**OpEx**

Domain 8.4の総括

ソフトウェア取得の安全性は**買う前に確認して終わり**ではない。**事業ニーズ・リスクの評価 → 契約に要件と検証権限を組み込む → 納品・受け入れの証拠を確認 → 稼働後の監視・変更・サポート・廃止まで管理する**ことでアシュアランスを維持する。OSS・COTS・クラウド・委託・孤立したシステム・M&Aのどれでも、**組織の説明責任と残余リスク判断は継続する。**

8.5 セキュアコーディングのガイドラインと標準を定義・適用する

1. コードの安全性とソフトウェア開発プロセス

教材読み上げの要旨

- ソフトウェア開発は業務・ミッションのニーズから始まり、機能・非機能要件、アーキテクチャ、サブシステム、コードへ落とし込まれる。
- 脆弱性はソースコードだけでなく、要件、設計、データシステム、Web環境、テスト、導入・利用のいずれにも生じうる。
- 再利用、機能追加・変更、リファクタリング、データモデリング、データ品質、セキュリティ実践が開発の品質を左右する。

補足：CISSPの判断

- 安全性は「最後にテストで足す」のではなく、全工程に組み込む（Secure SDLC）。
- 再利用したコードの弱点も再利用される。変更・リファクタリング後にはレビュー、変更管理、回帰試験が必要。
- **Refactoring**＝期待される外部の動作を変えずに内部構造を改善すること。セキュリティ変更や機能追加と同一ではない。
- **Data Quality**（データ品質）の不備は完全性・業務判断にも影響。型チェックはビジネスロジックの正しさを保証しない。

2. 手続き型とオブジェクト指向、および高・低水準言語

分類	主要な見方	代表的な言語・特徴	セキュリティの観点
手続き型（Procedural）	関数・手続きを組み立て、順序に沿って処理	C、Pascal、COBOL等（OOPと併用できる言語もある）	境界条件、入力検証、状態管理、メモリ管理
オブジェクト指向（OOP）	データ（状態）とメソッド（操作）をオブジェクトにまとめる	Java、C++、C#、Python、Ruby、Smalltalk、Delphi、Eiffel、Perl等	カプセル化、公開APIの認可、継承と多態性の影響
低水準（Low-Level）	CPU・命令・メモリに近い	機械語、x86/ARM Assembly	ハードウェアへの近さ、メモリ破壊等に注意
高水準（High-Level / HLL）	人に分かりやすい抽象度で処理を記述	C、Java、Python、Ruby等	可読性・レビュー支援はあるが、自動的に安全にはならない

注意：オブジェクト指向と手続き型は一律の二者択一ではなく、複数パラダイムを扱う言語もある。試験向けには、手続き型＝「処理の組み立て」、OOP＝「データと操作をまとめて扱う」という軸を優先。

3. ソースコードから実行可能プログラムへ

教材読み上げの要旨

- 最初期は開発者が機械語のバイナリ命令を直接作成した。
- アセンブリ言語は機械語を記号化して人間が扱いやすくした低水準言語。一般にはアセンブリの命令と機械語命令は比較的直接対応する。
- 高水準言語は複雑な操作を読みやすく簡潔に表現でき、標準のコーディング規則は後の検査・変更にも役立つ。

主要コード種別

用語	意味	区別のポイント
ソースコード（Source Code）	人間が記述・理解するプログラム本文	レビュー、SAST、変更管理の対象
オブジェクトコード（Object Code）	コンパイル／アセンブルにより得る機械語等の生成物	多くの場合、他のオブジェクトやライブラリとのリンクが必要

用語	意味	区別のポイント
実行可能コード（Executable）	対象環境で実行できる形のプログラム	署名、ハッシュ、配布と実行権限を管理
中間コード（Intermediate Code）	ソースとネイティブコードの間の表現	Javaバイトコード、.NET ILなど。VMやJITで実行
任意コード実行（Arbitrary Code Execution）	攻撃者が意図するコードを対象上で実行できる状態	侵害されたプロセスの権限が被害範囲を左右する

典型的なネイティブ・コンパイル：**ソースコード → コンパイラ → オブジェクトファイル → リンカ → 実行可能ファイル → CPUが実行**。すべての言語が同じ経路とは限らない。

最重要比較：アセンブラ・コンパイラ・インタープリタ

比較	アセンブラ	コンパイラ	インタープリタ
入力	アセンブリ言語	高水準言語（ほかの表現もある）	ソースコード・バイトコード等
主な役割	記号命令を機械語へ翻訳	コードを機械語または中間コードへ変換	プログラムを実行時に解釈・実行
生成物の典型	オブジェクトコード	オブジェクトコード、実行形式、中間コード等	独立した実行ファイルを必ずしも生成しない。実装によって中間コード等も使用
処理のタイミング	通常、実行前	AOTなら実行前／JITなら実行時	原則として実行時
CPUでの実行	生成されたネイティブ命令を直接実行	ネイティブ生成物はCPUが直接実行。中間コードは追加の実行基盤が必要	インタープリタ自体がCPUで動き、対象コードを解釈する。JITと併用する場合も
利点	ハードウェアに近い制御	最適化、事前チェック、高速実行が可能	試行・デバッグ・環境移植が容易な場合がある
例	x86／ARM Assembly	C、C++、Rust、Go	Python、Ruby（代表的実装）
セキュリティ	メモリ・命令操作の危険	コンパイラとビルド成果物の信頼	ランタイムの安全性・権限管理

混同注意：言語そのものをコンパイラ型／インタープリタ型と完全に固定するのは不正確。同じ言語に複数の実装方式があり得る。

中間コード・VM・JITの代表例

言語	分かりやすい代表経路	直前の記憶フック
C／C++	ソース→コンパイル・リンク→ネイティブ実行	事前コンパイル

言語	分かりやすい代表経路	直前の記憶フック
Python	ソース→バイトコード等→Python実行環境（一般的なCPython）	インタープリタ型の代表（内部処理は単純な逐次解釈だけではない）
Java	ソース→Javaバイトコード→JVM（解釈・JITなど）	バイトコード＋JVM
C#	ソース→CIL/IL→.NETランタイム（JITなど）	中間コード＋ランタイム
JavaScript	ソース→エンジン（解釈・JITなど）	インタープリタとJITの組合せ

4. プログラミング言語の世代（1GL〜5GL）

世代	主な名称	抽象化・特徴	具体例	重要な留意点
1GL	機械語	バイナリのCPU命令、オペコード	x86／ARMの機械語	マルウェアの逆解析・メモリ攻撃分析では現代でも重要
2GL	アセンブリ言語	記号によって機械命令を表現	x86 Assembly、ARM Assembly	ハードウェア・アーキテクチャへの依存
3GL	高水準言語	人間に読みやすい構文	C、C++、Java、Python、COBOL	実行方式との分類軸は別
4GL	第4世代／超高水準・結果指向	処理手順より得たい結果、レポート・アプリ生成	SQL、SAS、レポート生成ツール、教材で扱うGUI型生成環境	製品によってノーコード・ローコード、スクリプト言語の扱いは異なる
5GL	論理・制約ベース	論理規則・制約を与え推論させる	Prolog、Mercuryなど	自然言語インターフェースは歴史的説明の一例で、すべてが5GLではない

記憶フック：1GL＝機械、2GL＝アセンブリ、3GL＝高水準、4GL＝結果重視、5GL＝論理・推論。

1GLのセキュリティ上の用途

- 読み上げでは、マルウェアのリバースエンジニアリング、バイナリレベルでのメモリ／ストレージ違反やペイロード分析が例示された。
- 補足：マルウェアが必ず機械語を手書きして作られるわけではない。CISSPは解析技法の細部より、メモリ破壊、攻撃者制御のデータ、任意コード実行の関係を理解する。

4GL、ノーコード／ローコードとエンドユーザー開発

- 教材の例：ドラッグ＆ドロップ型のGUIで、専門言語の知識がないユーザーが業務フローやデータ分析アプリを作成できる。
- **読み上げで明示された懸念①**：利用部門ごとの大幅なカスタマイズや専門開発者以外によるアプリ作成で、統制・訓練・セキュリティレビューが十分にならない場合がある。
- 具体的なリスク：シャドーIT、公開設定の誤り、過剰権限、不適切なデータ連携、変更追跡不足、機密データの外部転送。
- 補足的統制：業務アプリの台帳、アクセス制御、DLP、承認されたコネクタ、変更管理、監査、開発者教育。

- **教材に「二つの主要な懸念」とあるが、今回の読み上げではもう一方が明確に完結していない。②は次回原文・読み上げで補完する。**

5. データ型の強制、データ隠蔽、コード隠蔽

用語	保護対象・目的	具体例	限界・注意点
Type Enforcement（データ型の強制）	型に適合しない操作や変換を検出・拒否	数値用の値に無関係な文字列を混ぜる操作を検出	ビジネスロジックの正しさは保証しない。暗黙変換の許容度は言語次第
Data Hiding（データ隠蔽）	内部データへの直接アクセス・変更を制限	銀行口座の残高をprivateにし、入出金メソッドから操作	オブジェクトのprivate属性はOSのアクセス制御と同義ではない
Code Hiding（コード／ロジック隠蔽）	コード・内部実装への不適切な参照・変更を制限	ライブラリのアクセス権、コードの完全性管理	コード署名は改ざん検知に役立つが、コードの機密性を保証しない

6. オブジェクト指向（OO/OOP）の構成要素と安全性

教材読み上げの要旨

オブジェクトは、関連するデータと、それを扱う一つ以上の**メソッド**のまとまりとして説明された。オブジェクト指向言語の例：Java、Python、C++、C#、Ruby、Perl、Smalltalk、Delphi、Eiffel。

概念	定義	セキュリティ上のポイント
カプセル化（Encapsulation）	内部の状態と操作をオブジェクトにまとめ、公開するインターフェースを制御	データ隠蔽。ただしメソッド内の認証・認可・入力検証も必要
継承（Inheritance）	親クラスの性質や操作を子クラスへ引き継ぐ	セキュリティ上の前提・不変条件が子クラスにも適合するか検証
多態性（Polymorphism）	同一の呼び出しでもオブジェクトの型に応じて異なる処理を行う	サブクラスのオーバーライド等で認可を迂回しないか確認
データ型（Types）	扱う値や操作の範囲を定める	型チェックと暗黙変換、境界条件を考慮
インスタンス化（Instantiation）	クラスなどから個別のオブジェクトを作る	生成時の初期権限・初期値の安全性

例：銀行口座オブジェクトの`balance`を内部データにし、外部からは`deposit`／`withdraw`のメソッドを通す。ただし、`withdraw`が呼び出し者の権限を検証しなければ、カプセル化だけでは安全ではない。

多重インスタンス化（Polyinstantiation）と推論攻撃（Inference Attack）

- **Polyinstantiation**：多段階セキュリティのデータベース等において、**同じキーや識別子の下で、機密区分ごとに異なる値・レコードのインスタンスを認める**考え方。
- **Inference**：許可された情報やシステムの応答から、直接閲覧できない機微情報を推測すること。

- 例：下位ユーザーが案件IDを登録しようとしたとき、上位分類に存在する同じIDとの重複エラーを返せば、機密案件の存在が推測される。多重インスタンス化は下位ビューに別インスタンスを持たせ、この存在の暴露を緩和し得る。
- ただし完全な防御ではなく、レコードの一貫性・管理の複雑性などに課題がある。
- **分類上の注意**：教材はOOの文脈で説明しているが、Polyinstantiationは一般的なOOP三大特徴ではなく、**多段階セキュリティ／データベース機密性**に関係する専門概念。

7. 分散オブジェクト指向のアーキテクチャ

教材読み上げに登場した用語

技術	名称・仕組み	記憶フック／位置づけ
CORBA	Common Object Request Broker Architecture：異なるプラットフォーム／言語の分散オブジェクト通信を仲介	異種環境。**ORB**（Object Request Broker）が呼び出しを仲介
Java RMI	Remote Method Invocation：Javaオブジェクトの遠隔メソッド呼び出し	Java間の遠隔呼び出し
EJB	Enterprise JavaBeans、現在はJakarta Enterprise Beansの名称	Javaのサーバー側コンポーネントモデル。CORBAやRMIと全く同じ分類ではない
DCOM	Distributed Component Object Model	Microsoft COMの分散機能

CORBAのセキュリティサービス：教材で挙げられた4領域

領域	目的	簡単な例
アクセス制御（Access Control）	呼び出し元が対象の機能・オブジェクトにアクセス可能か判断	ユーザー・サービスの認可
データ保護（Data Protection）	データの機密性・完全性を保持	保護された通信路、改ざん検知
否認防止（Nonrepudiation）	当事者が操作・通信を不当に否定できないようにする	証拠性ある記録・署名（要件と仕組みに依存）
監査（Auditing）	操作の追跡・検証	呼び出し・アクセスのログ記録・レビュー

CORBA実装で確認する事項（教材読み上げ）

- 対応するセキュリティ機能・セキュリティ構成要素（building blocks）。
- 管理者が組織のセキュリティポリシーを設定できるインターフェース。
- 対応するアクセス制御メカニズムの種類。
- 監査ログの取得対象、粒度、確認手段・ツール。
- **Common Criteria（CC）**等に関連するセキュリティ評価。**評価された製品の保証と、それを組み込んだシステム全体の安全性は別**。

分散システム共通の試験観点：ネットワーク境界を越えるので、通信相手の認証、操作ごとの認可、暗号化・完全性、監査、最小権限が必要。カプセル化だけでは足りない。CORBAが本試験で高頻度出題されると断定できる根拠は、今回の読み上げだけではない。

8. 「ソースコードを書くのは誰か」：ソフトウェアサプライチェーン

教材読み上げの要旨

ソースコードの作成・提供元と、その開発・品質保証・セキュリティプロセスへの信頼が、ITサプライチェーンのリスクに関係する。読み上げでは、大手ハードウェア・システムベンダーから、アプリケーション提供者、委託開発者、社内開発者、オープンソース、シチズンプログラマー、モバイルコードまでの層が列挙された。

ソフトウェアの提供・作成元	教材での位置づけ・主要なリスク	管理策（補足）
ハードウェア／システムベンダー	製品整合性と安全な開発プロセスを持つことが期待される	ベンダー評価、供給元の保証、ファームウェア更新、署名・完全性検証
大手アプリケーションベンダー	多数のユーザーへ商用ソフトを供給。利用者側は実装を直接確認しにくい場合も	サポート期限・パッチ・脆弱性情報・契約管理
サードパーティー開発者	カスタム開発・商用製品の調整。**安全性と保守の責任分担について合意して契約する**	セキュリティ要件、SLA、監査・評価権、納品物の検証、脆弱性対応契約
社内開発者	開発・運用のプロセスを自社が直接コントロールできる	SQA、セキュアコーディング、ソース管理、構成／変更管理、レビュー、テスト、説明責任
オープンソース	開発主体や維持管理の成熟度が多様	SCA、SBOM、依存関係・脆弱性・ライセンス・アップデート管理
シチズンプログラマー	専業開発者以外がローコード／ノーコードで業務アプリを作る	アプリ台帳、承認済みプラットフォーム、教育、権限制御、監査、DLP
モバイルコード	外部から取得・転送されたコードが実行される	出所・署名・完全性検証、サンドボックス、最小権限、実行制限

最重要な解釈上の注意：教材にある「下位の層ほど安全・信頼性が低下する」という説明は**リスクを考えるための教材上の傾向**であり、**OSSだから常に商用品より危険、社内開発だから常に安全**という絶対的な順位づけではない。信頼性は透明性、成熟度、開発プロセス、レビュー、保守、実際の統制に依存する。

サプライチェーンの保護範囲（補足）

調達・契約 → 設計／コード開発 → 依存コンポーネント → ビルド → 配布／署名 → 展開／運用 → 更新・廃止を一連の信頼チェーンとして管理する。ソースが適切でもビルドや配布環境が侵害されれば生成物に悪意ある変更が入り得る。実行成果物のハッシュ・署名、ビルド権限の最小化、監査、脆弱性修正なども重視する。

9. 直前暗記用：見分けの一問一答

問い	答え・連想
機械語は何世代？	**1GL**

問い	答え・連想
アセンブリ言語は？	**2GL**。アセンブラが機械語に変換
C、Java、Pythonの世代は？	**3GL**。実行方式の分類は別
SQL、レポート生成等は？	典型的には**4GL**（分類は文献差あり）
論理・制約に基づく言語は？	**5GL**、Prologなど
Javaがソースから生成する代表例は？	**バイトコード**。JVMが解釈・JIT等で実行
データと操作をまとめて外部公開面を限定する？	**カプセル化**
親クラスの性質を引き継ぐ？	**継承**
同じメソッド呼び出しで型ごとに挙動が変わる？	**ポリモーフィズム（多態性）**
同じキーで分類レベルごとに別の値？	**Polyinstantiation（多重インスタンス化）**
許可されたデータから秘密を推測？	**Inference Attack（推論攻撃）**
異種環境の分散オブジェクト通信？	**CORBA** ／ ORB
Javaの遠隔メソッド呼び出し？	**RMI**
Microsoftの分散COM？	**DCOM**
Javaサーバー側のコンポーネント？	**EJB**
CORBAセキュリティの四領域？	**アクセス制御・データ保護・否認防止・監査**
コードの出所・信頼性は何で判断？	**供給元の名称ではなく、検証可能な統制と継続管理**

10. 続きの読み上げで補完する項目

1. **8.5の未読部分すべて**（現時点で完了扱いにしない）。
2. 4GL／ノーコード開発の「二つのセキュリティ懸念」の**二つ目の教材説明**（読み上げでは未確定）。
3. ソフトウェア供給者分類の詳細、特にオープンソース・シチズンプログラマー・モバイルコードに関する後続説明。
4. 同じテーマの追加説明が出ても既存項目を削除せず、対応節を加筆し、最後に誤り・重複を整理する。

中間チェックポイント：ここまでが2026-10-28の8.5学習内容。次の読み上げは「ソフトウェアの提供元・サプライチェーン」の続きから。

8.5 補完統合｜攻撃ベクターまでの別版から保持した学習内容

> 追加入力ファイルに含まれていた8.5.1〜8.5.8の内容を、重複する記述も資料差分として保持して統合した。
> 後続のデータベース・Web・OWASP節も同一文書に収録する。

8.5 セキュアなソフトウェア開発に関するコーディング手法・脆弱性・攻撃ベクター【2026-10-28｜データベースの節に入る直前まで】

> **記録範囲**：今回の口頭学習で扱った、プログラミング言語、オブジェクト指向、供給網、入力検証、モバイルコード、隠れチャネル、オブジェクト再利用、および一般的な攻撃ベクター5項目。**次の「データベース」の説明はまだ開始していない。**以下は口頭で確認された教材の要旨と学習上の補足を整理したものであり、教材原文の逐語転記ではない。会話中の各分野で未確定の詳細は断定しない。

8.5.1 データ型、コードの隠蔽、言語の種類

テーマ	定義・仕組み	セキュリティ上の焦点
データ型の強制（Type Enforcement）	変数・引数等の型を制限し、許されない型や操作を拒否	型不一致、誤変換、型混同の一部を抑える。ただし入力の業務的意味や認可の確認は別途必要
データ隠蔽（Data Hiding）	内部データの直接操作を避け、定められたインターフェースから扱う	意図しない変更や情報への不正な到達を抑える。アクセス制御の代替ではない
コード隠蔽（Code Hiding）	実装の詳細を利用側に公開しない抽象化・カプセル化	誤用を減らせる場合があるが、「実装が非公開だから安全」とする隠蔽によるセキュリティとは区別
機械語（Machine Language）	CPUが直接処理する命令	ハードウェア依存、低水準で安全性に関する責任が大きい
アセンブリ言語（Assembly Language）	CPU命令に対応する記号的な言語。アセンブラで機械語へ変換	メモリ・レジスタの扱い、境界外アクセス等を理解する
高水準言語（High-Level Language）	人が理解・記述しやすい抽象化された言語	言語の型システム、ランタイム、メモリ管理などが一部の欠陥防止に役立つ
コンパイラ（Compiler）	ソースコードを別の実行形式・中間表現などへ翻訳	ビルド時チェックが可能な場合がある。翻訳方式だけで安全性は決まらない
インタプリタ（Interpreter）	実行しながらコードや中間表現を解釈・処理	実行時の入力・依存コード・権限の制御が重要

世代別の整理：1GL＝機械語、2GL＝アセンブリ言語、3GL＝一般的な高水準言語、4GL＝より宣言的・領域特化的な高水準言語、5GL＝制約・論理・問題記述などを重視する言語群として説明される。分類には幅があるため、個別言語名を世代のみで安全度ランキングにしない。

8.5.2 オブジェクト指向プログラミング（OOP）と関連技術

概念	要点	注意点・例

概念	要点	注意点・例
カプセル化（Encapsulation）	データと処理をオブジェクト内にまとめ、公開する操作を制御	内部フィールドを不用意に直接変更させない。ただし適切な認可は別に必要
継承（Inheritance）	既存クラスの性質・操作を引き継ぐ	上位クラスの不適切な機能や権限が下位クラスに波及し得る
ポリモーフィズム（Polymorphism：多態性）	同じインターフェースでも型によって異なる動作をする	意図しないメソッド実行や振る舞いの見落としに注意
複数インスタンス化	同じクラス定義から独立した複数オブジェクトを生成	インスタンスごとの状態・権限・データを混同しない
推論攻撃（Inference Attack）	直接閲覧できない情報を、公開された結果や関連情報から推定	統計や集計、検索結果などを組み合わせた機密情報の推測

分散オブジェクト技術の歴史的な例：CORBA、Java RMI、EJB、DCOM。分散環境でメソッドを呼び出す等の機能を提供するが、古典的・歴史的技術として扱い、現代の推奨標準だと決めつけない。CORBAのセキュリティサービスは、分散オブジェクト間の認証、アクセス制御、監査などの実装・運用に関わる話として整理する。

8.5.3 ソフトウェアの供給元・再利用に潜むリスク

供給元の分類として、大手・一般ソフトウェアベンダー、契約開発者／委託先、社内開発者、オープンソース、市民開発者（Citizen Developer）、ネットワークから取得されるモバイルコード等を確認した。**供給元の名称や規模だけでは安全性は保証されない。**再利用するモジュールについて、出所、署名・ハッシュ、脆弱性、依存関係、潜在的な悪性ロジック、更新・保守責任を検証する。

8.5.4 入力処理・メモリに関する代表的脆弱性

項目	主因と仕組み	影響例	防止・緩和
バッファオーバーフロー	許容される領域を超えるデータを書き込む等のメモリ境界違反	異常終了、データ破損、条件によって任意コード実行	長さ・境界チェック、メモリ安全な言語／API、保護機能、テスト
不正形式の入力（Malformed Input）	異常な形式・長さ・構造等の入力を送り、実装欠陥を突く	誤処理、障害、境界違反、注入の誘発等	入力検証、例外処理、ファジング、解析とテスト
不完全なパラメータ検証・強制	パラメータの型・書式・値域・長さ・意味・権限を十分確認しない	不正な金額・対象・操作の受け入れ、データ破損等	一貫した検証と強制、ホワイトリスト的な許可条件、サーバー側認可
悪意ある入力による注入	入力データがコード・コマンド・クエリ等として誤って解釈される	不正操作、データ漏えい・改ざん	コードとデータの分離、パラメータ化、文脈に適したエンコード／無害化
型の不一致・型混同	不正な型の解釈、暗黙変換等	予期しない分岐や計算・認可の誤り	強い型指定、型検証、危険な変換の排除

区別：Malformed Inputは「攻撃者が与える不正な入力・試行」、不完全なパラメータチェックは「それを受け止める側の検証不備」という関係。**境界値チェックはパラメータ検証の一部でしかない。**例えば振込額は整数であること、1〜100万円の範囲であること、残高や利用上限に整合すること、送金主体に権限があることを別々に検証する。

防御的プログラミング（Defensive Programming）：異常・失敗・悪用を前提に、安全な既定値、入力検証、例外処理、最小権限、安全側への失敗、ログ、テストなどを組み込む。**Allowlist（許可リスト）は明示的に許容する対象を定める方式**、Blocklist（拒否リスト）は既知の禁止対象を列挙する方式。許可リストは未知の危険な入力も制限しやすいが、いずれも文脈に適した設計が必要。

8.5.5 実行可能コンテンツとモバイルコード固有の脆弱性

- **実行可能コンテンツ**：受け取った環境で実行され得るプログラムを含むコンテンツ。**モバイルコード**：ネットワーク等を通じて移され、受け取り側の環境で動作するコード。
- **教材で挙がった主なリスク＝CIAの侵害**：機微情報の開示（機密性）、データの損傷・改ざん（完全性）、リソースを浪費し正当な利用を妨げるDoS（可用性）。
- **対策① サンドボックス**：実行環境・権限・ファイル／ネットワークアクセスを制限し、被害を局所化する。
- **対策② デジタル署名・証明書を用いた出所検証**：署名者と署名後の完全性を適切に検証する。**署名があることはコードの無害性を保証しない。**
- 対策は相補的であり、出所検証＋実行時の制限を重ねる。JavaScriptのXSSや意図しない外部コードの実行などでは、適切な出力エンコード、CSP、最小権限も検討する。

8.5.6 隠れチャネル（Covert Channels／Covert Paths）とコントロール

定義：本来情報伝達用ではない仕組み・資源を使い、許可されていない情報のやり取りを行うチャネル。
APT（Advanced Persistent Threat：高度持続的脅威）のC2（Command and Control：指揮統制）通信などに悪用され得る。ただし、すべてのAPTやC2通信に隠れチャネルが使われるわけではない。

種別	英語・略語	情報の表現方法	具体例	主な統制
隠れストレージチャネル	Covert Storage Channel（**CSC**）	共有リソースの状態・データを書き換えて信号化	ファイル属性や共有状態の値を切り替え、別プロセスが観測	アクセス制御、最小権限、資源・プロセス分離、共有状態の制限
隠れタイミングチャネル	Covert Timing Channel（**CTC**）	処理タイミング、通信間隔、リソース利用時間を信号化	CPU負荷を変化させ、その処理時間の差を別プロセスが観測	資源隔離、タイミング挙動の均一化・ノイズ、監視

隠れチャネルコントロール（Covert Channel Controls）：隠れた経路を特定・分析し、可能なものは排除し、排除困難なものは帯域幅を抑え、観測・検知する。**帯域幅（Bandwidth）**は単位時間当たりに秘密裏に伝送できる情報量。低帯域でも機密情報は流出し得るため、「帯域幅が低い＝安全」とまではいえない。

対比：バックドア＝正規の認証・アクセス制御を回避する隠しアクセス経路／隠れチャネル＝正規に認められていない情報伝達経路。機能が重なることはあっても同義ではない。

8.5.7 メモリまたはその他のオブジェクトの再利用（Memory / Object Reuse）

現象：ある利用者・プロセスが使ったメモリ、ファイル、ディスク領域、その他のオブジェクトを別の利用者へ再割り当てした際に、**前の内容が残留（Data Remanence）し、次の利用者へ漏れる**問題。

例：プログラムAがメモリにパスワードを保持→領域を解放するが初期化しない→プログラムBが同じ領域を取得し、残存内容を参照できてしまう。

対策：適切な再割り当て時の初期化、必要なデータ消去、メモリ・プロセスの分離、オブジェクトの適切な破棄・再利用規則。「解放された＝データが消去された」は誤り。

8.5.8 その他の一般的なソフトウェア攻撃ベクター：5項目比較【今回の記録終点】

No.	名称	仕組み・弱点	例	代表的対策
1	**ソーシャルエンジニアリング（Social Engineering）**	人の心理・信頼・権限認識を悪用	管理者を装い認証情報を聞き出す	教育と訓練、本人確認手順、MFA、職務分離
2	**TOCTOU（Time of Check to Time of Use）**	検査時点と使用時点の間に対象が変更され、検査結果が有効でなくなる。競合状態の一類型	安全なファイルだと確認後、開くまでに攻撃者が差し替える	原子的操作、適切なロック、ファイル記述子を用いた安全な操作、再検証
3	**競合状態（Race Condition）**	並行実行の順序・タイミングにより予期せぬ結果が起こる広い問題	在庫1個に同時注文が通る、残高更新が失われる	同期、排他制御、原子性、トランザクション／同時実行制御
4	**Between-the-Lines攻撃**	正規の通信セッションの空き時間・隙間を悪用して、攻撃者が不正な通信・操作を挿入する古典的な攻撃説明	正規セッション中の入力がない隙に、別のコマンドを紛れ込ませる	セッション保護、強固な認証、通信完全性の検証、タイムアウトと監視
5	**トラップドア／バックドア（Trapdoor / Backdoor）**	通常の認証・アクセス制御を回避する隠し経路・機能	残された秘密の管理者用ログイン手段	コードレビュー、変更統制、不要機能の廃止、アクセス監査、最小権限

混同防止とCISSP暗記

- **Race Conditionは広い分類、TOCTOUは「確認→使用」の間に変化する特定の競合問題。**
- **Between-the-Lines**は古典的な通信セッションへの割り込みとして理解し、一般の盗聴やすべてのセッションハイジャックと機械的に同一視しない。
- **ソーシャルエンジニアリング**＝人を利用／**TOCTOU**＝確認と使用の時間差／**Race Condition**＝並行処理の順序／**Between-the-Lines**＝正規通信の隙間／**Backdoor**＝通常制御を迂回する裏口。
- 技術的な欠陥はレビュー・セキュア設計・検証で減らし、人に対する攻撃には教育・プロセス・認証強化を組み合わせる。単一対策での完全防止を想定しない。

> **現在位置（2026-10-28）**：8.1完了、8.2は既存収録範囲、8.3完了、8.4完了、**8.5は「その他の一般的なソフトウェア攻撃ベクター」の5項目まで**。次回は**8.5のデータベース関連項目**から再開。ここまでの旧マスター記述は削除していない。

8.5 続編｜データベース・データウェアハウス・Webアプリケーションの脅威と保護【2026-10-28 統合追記】

> **位置づけ**：以下は、上記マスターの8.5既存内容を残したまま、今回の教材読み上げと学習会話を整理した続編。教材で挙がった論点と、理解のための具体例・セキュリティ補足を区別する。従前の「8.5の未読部分」の注記は、その時点の学習進捗を示す履歴として保持する。

11. データベース／データウェアハウス・アーキテクチャへの脅威

11.1 用語と環境

- **エンティティ（Entity）**：顧客・注文・商品など管理対象の実体。**リレーションシップ（Relationship）**：顧客と注文の紐づきなど、実体間の関連。**データモデル**は両者や制約を表現する方法。
- **データベース（DB）／DBMS**：データを体系化して保持し、DBMSが保存・検索・更新・権限管理・復旧などを実行する。
- **データウェアハウス（DWH）**：複数の業務システムのデータを統合・整備し、主に分析・意思決定に使う。典型的には事前に整理したスキーマを重視（schema-on-write）。
- **データレイク**：構造化・半構造化・非構造化データを原形式に近い形でも蓄積し、利用時に解釈することが多い（schema-on-read）。分類・品質管理を怠ると「データスワンプ」化する。
- **ビッグデータ**：量（Volume）、速度（Velocity）、多様性（Variety）などの特性を持つデータや処理課題。**DWH／レイクは保管・分析基盤、ビッグデータはデータ・処理の特性**であり同義ではない。

11.2 攻撃サーフェスにあるストレスポイント

脅威（教材）	仕組み・具体例	防御の視点
集約と推論（Aggregation / Inference）	個別には公開可能な統計を組み合わせ、秘密情報を推測する。複数の集計表から特定人の属性を絞り込む	クエリ・出力制御、最小集計件数、機微データ分類、推論リスク評価
バイパス攻撃	意図された認証・認可のチェックを迂回する	すべての入口に一貫した認可、完全仲介
アクセス制御に使うDBビューの侵害	制限付きビューでは隠れる列を、元表への直接権限や不適切なビューを通じて参照する	基表の権限撤廃、ビュー定義・所有者権限の監査
代替アクセスルートの悪用	管理用API、バックアップ、レプリカ、ETL、エクスポート等の別経路を攻撃	すべての経路を資産把握し、同等の認可・監査を適用
データ汚染	不正・誤ったデータがDB、DWH、分析結果に広がる	入力検証、データ品質・出所追跡、隔離と完全性確認
データ傍受	通信やDB接続からクエリ・認証情報・データを盗聴	TLS、証明書検証、暗号鍵管理
クエリ攻撃	SQLインジェクションや意図しない大量／高負荷クエリ	パラメータ化クエリ、最小権限、制限と監視
物理アクセス・サーバーへの直接アクセス	DBファイル、ストレージ、管理コンソールへの直接接触	物理保護、ディスク暗号化、OS権限、管理経路制限

脅威（教材）	仕組み・具体例	防御の視点
TOCTOU（Time of Check to Time of Use）	権限や状態の**確認時**と**使用時**の間に対象が変わる競合状態	適切なロック・トランザクション、アトミック操作、再検証
Webベース攻撃／不正アクセス	Webアプリ/API経由の認可不備、セッション悪用、注入等	アプリ認可、入力検証、WAF等の多層防御、監査

CISSP論点：DWHやデータレイクに集めたデータは、個別データセットの機密性だけでなく、**統合・分析結果からの推論**も評価する。

12. DBMSアーキテクチャと主要なデータベースモデル

12.1 DBMSに期待する基本的な能力

教材で列挙：**トランザクションの永続性、フォールトトレラントなリカバリ、複数ユーザー間での共有、セキュリティコントロール**。データの所在が仮想ボリュームや別拠点に分散していても、アプリケーションに一定の操作モデルを提供する。

ACID（多くのトランザクションDBで重要）：

特性	内容	例
Atomicity（原子性）	一連の変更を全部確定、または全部取り消す	注文だけ登録して在庫更新に失敗したらロールバック
Consistency（一貫性・整合性）	定義された整合性制約を守る	外部キー・CHECK制約を破らない
Isolation（分離性）	並行トランザクションの干渉を分離レベルに応じて制御	同時購入で在庫数の競合を抑える
Durability（永続性）	コミットした変更は障害後も維持	障害後に確定済み取引を復旧

※ACIDはDBの**トランザクション整合性**を扱う性質で、認証・暗号化そのものを意味しない。

12.2 データベースモデルの進化：比較

モデル	データ表現・特徴	具体例	向く／注意する点
階層型（Hierarchical）	親子の**木構造**。典型的には子レコードに親が一つ	企業→部署→社員の階層、IBM IMS	1対多に自然。多対多の関連表現は複雑になりやすい
ネットワーク型（Network）	レコードを**網目状**の関連で結び、複数の親・所有関係を表現可能	CODASYL系モデル、IDMS	複雑な関係を辿れるが、アクセス経路・保守が複雑になりやすい
リレーショナル型（Relational / RDBMS）	**表（リレーション）**、行（タプル）、列（属性）で扱い、キー・制約・集合的操作を使う	PostgreSQL、MySQL、SQL Server、Oracle Database	SQLで柔軟に検索・結合できる。権限・制約・トランザクション管理が重要

モデル	データ表現・特徴	具体例	向く／注意する点
オブジェクト指向型（Object-Oriented / OODBMS）	データと振る舞いを**オブジェクト**として保存。識別子、クラス、継承等を利用	ObjectDB、GemStone/S	複雑なオブジェクト構造に適する場合。メソッド実行権限や継承による情報露出に注意

※音声の「回想型」は文脈から**階層型**として整理。

12.3 リレーショナルモデルの基本3要素

1. **データ構造**：リレーション（表）とその属性（列）・タプル（行）。
2. **整合性ルール**：属性の許容値（ドメイン）と組合せに関する制約。主キー・外部キー・NOT NULL・CHECK等が実装例。
3. **データ操作**：関係代数／関係論理を基盤とする検索・集合操作や更新。選択・射影・結合などが重要で、SQLが代表的な操作手段となる。

用語	意味	例
テーブル／リレーション	行と列の論理的なデータ集合	`Customer`表
タプル／レコード	表の1行	顧客ID=101、氏名=佐藤
属性／フィールド	表の列	顧客ID、氏名、住所
ドメイン	属性が取り得る値の範囲	年齢は適切な整数の範囲
主キー（PK）	行を一意に識別するキー	顧客ID
外部キー（FK）	別表のキーへの参照と参照整合性	注文の顧客ID→顧客表

13. データ整合性、CRUD、トランザクション

- **CRUD**：Create（作成）／Read（参照）／Update（更新）／Delete（削除）。アプリからの単一操作でもDBMS内部では複数のOS・ストレージ操作を伴うことがある。
- **トランザクション**：業務上、不可分に扱いたい複数のDB操作をまとめる処理単位。例えばECで「注文レコード作成＋在庫減算」を一つのトランザクションにする設計がある。
- **分散配置**：データが複数のボリューム・ノード・拠点に存在していても、DBMSが論理的な操作を提供する。整合性・障害復旧・遅延への注意が必要。
- **同時実行制御**：ロックやMVCC、分離レベルによって競合・ダーティリード・ロストアップデート等を制御する。

注意：実際のオンライン購入全体は、決済、配送、外部サービス等を含み、一つのDBトランザクションに収まらないこともある。その場合はSaga等の設計や補償処理が必要になる。

14. データベースインターフェース言語：SQLとXML

14.1 SQLの中心概念

概念	意味	例・セキュリティ注意

概念	意味	例・セキュリティ注意
スキーマ（Schema）	DBの論理構造・オブジェクト定義（DBMSにより名前空間の意味も持つ）	表、列、制約等。定義変更権限を制限
テーブル（Table）	行・列で保持する基本的なデータ構造	`Orders`、`Customers`。最小権限
ビュー（View）	SQLの検索定義から提供される仮想的な表（実体化ビューを除く）	給与列を除いて表示。元表への直接権限で迂回されないよう制御

14.2 SQLのサブ言語

区分	正式名称	役割	代表的な文
DDL	Data Definition Language	DBオブジェクト・構造の定義・変更	`CREATE`、`ALTER`、`DROP`
DML	Data Manipulation Language	データの検索・追加・変更・削除	`SELECT`、`INSERT`、`UPDATE`、`DELETE`（SELECTをDQLに分ける分類もある）
DCL	Data Control Language	権限管理	`GRANT`、`REVOKE`

補足：`COMMIT`、`ROLLBACK`などのトランザクション制御文は**TCL**と分類することがある。SQLの主要区分は教科書・DBMSで分類差がある。

14.3 マークアップ言語とXML

- **マークアップ言語**：タグ等で文書やデータの構造・意味を記述するための形式。一般にDBへのアクセス言語と同義ではない。
- **XML（Extensible Markup Language）**：利用者定義のタグで階層的な構造化データを表す。設定、データ交換、システム連携等で使用。XML Schema（XSD）等で構造・型を検証できる。
- **主なリスク**：外部エンティティ参照を不適切に許可した場合の**XXE**、巨大・再帰的な展開による資源消費、取り込みデータの不適切な信頼。
- **対策**：安全なXMLパーサー設定、不要な外部エンティティ・DTDの無効化（要件に応じ判断）、入力のサイズ・深さ制限、スキーマ検証、最小権限。

15. アプリケーションからのデータベース接続

技術	正式名称	役割・特徴	典型例
ODBC	Open Database Connectivity	ドライバーを通じて異種DBMSへの共通的なアクセスを提供	BIツール／Excel→業務DB
JDBC	Java Database Connectivity	JavaからSQL DBにアクセスするAPI。JDBCドライバーを利用	Java業務サービス→PostgreSQL
OLE DB	Object Linking and Embedding, Database	MicrosoftのCOMを基盤とするデータアクセス技術。RDBMS以外のソースも扱える	従来のWindows／Microsoft系アプリ

技術	正式名称	役割・特徴	典型例
ADO	ActiveX Data Objects	接続・コマンド・レコードセット等をオブジェクトとして操作する高水準のデータアクセスAPI	VBScript／Classic ASPのDBアクセス
API	Application Programming Interface	アプリケーション間で機能／データを利用する境界。DB接続APIより一般的な概念	REST API経由で注文を照会

区別：ODBC／JDBC／OLE DBは主に接続・データアクセスの仕組み。ADOはOLE DBなどを利用する上位層のオブジェクトAPI。**ADO.NETは従来のADOとは別体系**。

15.1 インターネット経由のDBアクセスと階層型アプローチ

典型的な構成：**ブラウザー → Web/API層 → 業務ロジック層 → DB層**。インターネット利用者がDBへ直接接続する構成を避け、アプリを介して認証・認可・入力検証を行う。アプリ層のDB接続資格情報は機密管理し、DBアカウントには最小権限を付与する。内部通信も必要に応じてTLSで保護する。

階層型アプリケーションアプローチ（音声の「回想型」は文脈上こちらと解釈）：表示（プレゼンテーション）／業務処理（ビジネスロジック）／データアクセスを分離して責務と信頼境界を明確化する。階層化だけで認可が自動的に成立するわけではない。

16. メタデータと分析：OLAP・データマイニング・KDD

- **メタデータ**：データについて記述するデータ。列名・データ型・作成日時・所有者・分類・出所・変更履歴など。DBの構造や情報の意味を推測させる場合があり、メタデータ自体のアクセス制御も重要。

手法	役割	具体例	主要リスク
OLAP（Online Analytical Processing）	集計軸を切り替える**多次元分析**。スライス、ダイス、ドリルダウン等	地域×商品×月の売上を集計	小集団の結果や複数集計からの推論
データマイニング	大量データから傾向・関係・パターンを見つける手法	併売商品、異常取引パターンの発見	隠れた機微属性の推定、データバイアス
KDD（Knowledge Discovery in Databases）	データ選択→前処理→変換→**データマイニング**→評価・解釈など、知識発見の一連のプロセス	顧客行動を分析し、意思決定に活用	データ出所・品質・プライバシー・出力の管理

関係：データマイニングは通常**KDDの工程の一つ**。OLAPは集計・分析の技術であってKDD全体の別名ではない。OLTP＝業務取引の記録・更新、OLAP＝分析・意思決定支援。

17. Webアプリケーション環境：脆弱化要因

教材で列挙した古典的な設計・実装の問題を、セキュリティ上の理由と併せて整理。

脆弱化要因	問題となる理由・例	防止策

脆弱化要因	問題となる理由・例	防止策
アクセス制御・認証・資格情報管理を独自に再発明	権限判定やセッション管理に漏れを作り込む	実績ある認証・認可基盤／標準を採用、設計レビュー
暗号を独自設計・実装	暗号アルゴリズム・鍵管理・乱数で致命的誤り	標準方式と実績あるライブラリ、鍵管理
秘密情報やアクセス制御情報をハードコード	スクリプト、ページ、モバイルコードやリポジトリから露出	シークレット管理、サーバー側認可、コード検査
管理者・ユーザー・ホスト・ネットワーク権限の管理不足	過剰権限、未承認管理アクセス	最小権限、MFA、許可送信元の限定、権限棚卸し
入力規則・長さ・型・範囲のチェック不足	注入攻撃、想定外処理、データ破壊	サーバー側入力検証、パラメータ化クエリ、適切な出力エンコード
十分なセキュリティ評価をせず本番公開	欠陥・試験用設定・未修正問題を公開	SDLCのレビュー、セキュリティテスト、承認ゲート

17.1 主なWeb脅威と攻撃対象領域

- **DoS／DDoS**：負荷や資源枯渇によりサービスの可用性を損なう。DDoSは複数の送信元から分散して攻撃。CDN、レート制限、容量・防御サービス等を組み合わせる。
- **MITM（中間者攻撃）**：通信に介入して盗聴・改ざんを試みる。正しく検証されたTLS、証明書管理が重要。
- **Webの広い攻撃対象領域**：公開ページだけでなく、API、スクリプト、依存ライブラリ、Web・アプリ・DBサーバー、管理画面、ホスト、ネットワーク等を含む。資産と信頼境界を把握する。
- その他、SQLインジェクション、XSS、セッション乗っ取り、認可不備、設定ミス等にも注意する。

18. Webアプリケーションの保護戦略・コントロール

18.1 保護戦略（教材列挙を分類）

領域	コントロール	意図
保証・構成	Webサーバーのセキュリティ評価、OSハードニング、ベンダーパッチとアラート監視	既知脆弱性・不要機能を減らす
検査・検知・防御	Web／ネットワークの脆弱性スキャン、IDS／IPS、アプリケーションプロキシ／WAF	攻撃の検知・阻止と継続的改善
露出の最小化	不要な文書・ライブラリ・機能を削除または無効化、管理インターフェースを保護	攻撃面を減らす
管理アクセス	開発・サポート担当へのアクセス限定、許可されたホスト／ネットワークのみ接続、強固な本人認証	管理機能への不正アクセス防止
資格情報保護	秘密情報をアプリ内にハードコードしない、証明書等の信頼できる認証方式、秘密情報の安全な保管	資格情報流出・なりすましを防ぐ
監査・通信	ロックアウト等の適切な認証防御、拡張ログ・監査、認証トラフィックの暗号化	攻撃抑制・追跡・盗聴対策

領域	コントロール	意図
設計分離	UI／業務ロジック／データアクセス・保存の適切な分離	境界ごとの認可・最小権限・多層防御

18.2 Webシステムで「行うべきこと」

- **暗号化は業界で認められた標準**を満たし、実績のある実装と安全な設定を選ぶ。
- **コード／製品ベンダーのパッチとセキュリティ警告**を追跡し、影響を評価して更新する。
- **重要トランザクションと業務マイルストーンをログに記録**し、ログの機密性・完全性・保存を保護する。
- **例外を適切に処理**して、予期しないエラー時も安全側に失敗させる（Fail Secure）。内部情報をエラーメッセージで露出しない。

18.3 Webシステムで「してはいけないこと」

- **認証済みの機密ページを無制御にキャッシュしない。**機密情報を含む応答は用途に応じ `Cache-Control: no-store` 等で制御する（すべてのHTTPSページが必ずキャッシュ不可という意味ではない）。
- **クライアントからの値を信頼しない。**ブラウザの検証だけに依存せず、サーバー側で入力と**認可**をチェックする。
- **他サーバー・取引先・内部アプリの別コンポーネントから来た値も自動的に信頼しない。**内部通信も信頼境界を意識し、送信元認証、検証、アクセス制御を行う。

19. OWASPの整理とCISSPでの着眼点

OWASP（Open Worldwide Application Security Project）は、Web・ソフトウェア・APIの安全性向上のためのコミュニティ／非営利活動。単一の「フレームワーク製品」ではなく、複数のガイド、プロジェクト、成熟度モデル等を提供する。

OWASPの代表的成果物	目的	CISSPの覚え方
OWASP Top 10	Webアプリの主要なセキュリティリスクを整理する啓発・優先検討の指針	**代表的リスクを知る。**完全な診断項目リストではない
ASVS（Application Security Verification Standard）	アプリセキュリティ要件と検証項目を体系化	**何を満たすか／どう検証するか**
OWASP Testing Guide（WSTG）	Webアプリのセキュリティテストの進め方・観点	**どうテストするか**
OWASP SAMM	組織のソフトウェアセキュリティ活動の成熟度を評価・改善	**組織的なセキュア開発能力を高める**
OWASP Cheat Sheet Series	認証、入力検証、セッション等の実装ガイド	**安全な実装方法の参照**

典型リスク：アクセス制御の不備、暗号の誤用、インジェクション、不安全な設計、設定ミス、脆弱・古い部品、認証の欠陥、ソフトウェア／データの完全性不備、ログ・監視不足、SSRF等。OWASP Top 10の**分類名・順位は版によって変わる**ので、試験教材の版と区別する。

19.1 最終整理：リスク → 最初に考える統制

設問の手がかり	着眼点
分析結果を組み合わせると機密が判明する	**集約・推論**とクエリ結果／最小集計単位の制御
元表を参照してビューを迂回	**全経路で一貫したアクセス制御**
注文と在庫更新を一貫した業務操作として扱う	**トランザクション、ACID**
表／行／列、参照整合性	**リレーショナルモデル**
DDL／DML／DCL	**構造／データ操作／権限**
JavaからDB接続	**JDBC**（ODBCは汎用、OLE DB／ADOは従来Microsoft系）
OLAP／マイニング／KDD	**多次元分析／パターン発見／知識発見の全プロセス**
Webアプリで独自暗号・独自認証	**実績ある標準とライブラリを使用**
Webを本番に急いで投入	**セキュリティ要件、レビュー、試験、リスクに応じた承認**
API経由で他人の情報にアクセス	**オブジェクト単位も含むサーバー側認可**
セキュリティテストの体系的観点	**OWASP ASVS／WSTG**

CISSPの基本判断軸：技術を暗記するだけでなく、**守るべき情報・業務リスクを特定 → 信頼境界と全アクセス経路を把握 → 最小権限・標準化された統制を設計 → テスト・監査・継続的管理**へつなげる。

最新の学習再開位置（2026-10-28）：8.5の「データベース・データウェアハウスの脅威」から「Webアプリケーション環境・OWASP」まで整理済み。次回は教材のこの次の項目から再開する。

8.5.16 悪意のあるソフトウェア（Malware）と関連攻撃【今回の新規読み上げ】

教材読み上げの要旨

ソーシャルエンジニアリング、ウイルス（ファイル感染型・ブートセクター感染型・システム感染型・コンパニオン・電子メール・マルチパーティ・マクロ・スクリプト）、ワーム、デマウイルス、トロイの木馬、リモートアクセス型トロイの木馬（RAT）、ボットネット・ゾンビ・DDoS、論理爆弾、スパイウェア、アドウェア、いたずら（Pranks）、ランサムウェア（環境寄生型、マルウェアを伴わない身代金攻撃）を列挙。

主要な種類：定義・特徴・具体例（補足）

区分	仕組み・特徴	具体例・見分け方
ソーシャルエンジニアリング	人間の心理や信頼を悪用する攻撃手法。マルウェアそのものではない	管理者を装って添付ファイルを開かせる
コンピューターウイルス	通常、宿主となるファイルや領域へ寄生し、実行などを契機に複製・感染	感染した実行ファイルを開くと別ファイルへ広がる
ワーム（Worm）	宿主ファイルへの寄生を要せず、独立して自己複製・拡散する	脆弱な端末をネットワーク越しに次々感染させる

区分	仕組み・特徴	具体例・見分け方
デマウイルス（Virus Hoax）	存在しない脅威を装う虚偽の警告。通常は悪性コードそのものではない	「このファイルを削除しないと感染する」と偽情報を拡散
トロイの木馬（Trojan Horse）	正規・無害に見せかけて不正機能を実行する。ウイルス／ワームのような自己複製は定義上必須でない	偽の便利アプリが認証情報を窃取
RAT（Remote Access Trojan）	遠隔から不正に端末を操作するためのトロイの木馬	攻撃者が感染端末のファイルや画面へアクセス
論理爆弾（Logic Bomb）	日時や特定の条件で不正動作する仕掛け	退職日の翌日にデータ削除が始まる
スパイウェア（Spyware）	利用者の活動・データを秘密裏に収集	キー入力や閲覧情報を送信
アドウェア（Adware）	広告表示を目的とするプログラム。すべてが悪性とは限らない	意図せぬ広告や追跡を強制する不審なソフト
いたずら（Pranks）	迷惑・驚かせる演出などを目的とするプログラムや行為。必ずしも自己複製しない	偽警告画面を表示するジョークプログラム
ボット（Bot）	自動処理を行うプログラム。正規利用もある	遠隔の指示に従う感染端末上のボット
ボットネット（Botnet）	ボット化した複数端末が攻撃者の指揮下にある集合	多数の端末から同時にDDoS
ゾンビ（Zombie）	攻撃者の遠隔制御下で動く侵害端末	本人に無断で攻撃通信を送信するPC
DDoS	分散型サービス拒否。攻撃手法でありマルウェアの種類ではない	ボットネットによる大量通信でサービス停止
ランサムウェア（Ransomware）	データ暗号化・窃取・業務妨害等で身代金を要求	ファイルを暗号化し復号料を要求

ウイルスの分類：感染対象・伝播形態を軸に整理（補足）

種類	特徴	具体例・識別点
ファイル感染型	実行ファイル等に感染	実行したEXEを通じて感染拡大
ブートセクター感染型	ブート領域に感染し起動に関与	感染したメディアから起動
システム感染型	OSのシステムファイル等を感染対象とする分類	システムコンポーネントを書き換える
コンパニオンウイルス	正規プログラムと紛らわしい別ファイル等を置き、先に実行される仕組みを悪用	同名に近い実行ファイルで偽装

種類	特徴	具体例・識別点
電子メールウイルス	メール添付・リンク等を感染経路として悪用	添付を開いた際に感染（メールは伝播経路）
マルチパーティ（Multipartite）	複数種類の感染対象・方式を組み合わせる	ブート領域とファイルの両方に感染
マクロウイルス	文書等のマクロ機能を悪用	Officeマクロを有効化して感染
スクリプトウイルス	VBScript／JavaScript等のスクリプトを利用	悪意あるスクリプト実行で感染・拡散

分類の注意：この表は「感染場所」「実装技術」「配布経路」など異なる切り口が混在する。例えばメールで届くマクロウイルスは両方に当てはまる場合がある。

ランサムウェアの発展と身代金攻撃（補足）

- **環境寄生型（Living off the Land）**：OSや組織の正規ツール・管理機能を悪用して攻撃を進める手口。ランサムウェアでも活用されるが、独立したランサムウェアの標準分類名とは限らない。例：侵害済み権限と管理コマンドを使いデータへアクセスする。
- **マルウェアを用いない身代金攻撃**：データを盗み出し、公開すると脅すなど、必ずしも暗号化マルウェアを使わない恐喝。暗号化被害を伴わなくても重大な機密性リスクがある。
- **二重恐喝**：暗号化と情報公開の脅迫を組み合わせる例。バックアップだけでは情報漏えいへの対応にならない。

CISSP試験での識別ポイント・保護戦略（補足）

問題文の鍵	結び付く概念／対策
宿主に寄生して自己複製	ウイルス。実行制御、EDR、パッチ、検疫
自律的にネットワークで拡散	ワーム。ネットワーク分離、脆弱性修正、監視
正常なアプリを装う	トロイの木馬。供給元確認、最小権限、アプリ制御
遠隔操作される侵害端末	RAT／ボット／ゾンビ。端末隔離、認証・権限確認、通信監視
大量端末によるサービス妨害	ボットネットを用いたDDoS。レート制御、上流での防御、冗長化
特定日や条件で発火	論理爆弾。職務分離、変更管理、コード審査、監査
データを暗号化して金銭要求	ランサムウェア。復元可能なバックアップ、セグメンテーション、最小権限、訓練
データ公開の脅迫、暗号化なし	マルウェアなしの身代金攻撃。データ保護、侵害調査、インシデント対応
警告を拡散させるが実際のウイルスはない	デマウイルス。信頼できる情報源の検証と啓発

CISSPの管理視点：単一製品による検知だけでなく、資産・脆弱性管理、最小権限、ネットワーク分割、バックアップ**復元テスト**、監視・ログ、利用者教育、インシデント対応計画を多層的に組み合わせる。感染時は封じ込め・証拠

保全・復旧の優先順位を組織の計画に従って判断する。

> **現在位置（2026-10-28）**：8.5「悪意のあるソフトウェア（マルウェア）」のランサムウェアと非マルウェア型恐喝までを統合。次の読み上げはこの続きから。

8.5.17 セキュアコーディングガイドラインと標準【2026-10-28 読み上げ追加】

教材読み上げの要旨

- **強い型付け・型安全なプログラミング言語**を選ぶことにより、データ型やデータ構造の扱いを厳格にし、型の誤用に起因する欠陥を減らせる。
- **ポインターの使用を制御・制限**し、危険なメモリ参照・不正な領域アクセスなどのリスクを低減する。
- **信頼済みライブラリやコード断片であっても、再利用は無条件に認めない。** レビューや利用条件の確認を求める。
- コードスニペット再利用には次の二つの落とし穴がある。① 開発者が**何をするコードか、どう動作するか正確に理解していない**。② そのコードが**説明どおり正しく動くとは限らない**。
- セキュアコーディング標準・ガイドラインの遵守には**管理的側面と技術的側面**が必要。組織は選定した標準、フレームワーク、ライブラリ、スタイルを開発者へ指示し、理解・動機付け・実践を促す。

補足：管理と技術の区別

側面	何をするか	具体例
管理的統制	安全な開発のルール・責任・教育・例外承認を定めて運用する	セキュアコーディング規約、使用可能なライブラリの承認一覧、教育、ピアレビュー必須化
技術的統制	ルール違反や脆弱性を自動的・具体的に防止、検出する	型チェック、メモリ安全機能、Lint・SAST、SCA、CI/CDの品質ゲート、単体・回帰テスト

CISSP識別ポイント： 型安全やポインターの制限だけでは業務ロジックや認可の欠陥は防げない。再利用コードは**出所・仕様・動作・ライセンス・脆弱性**を確認し、開発者任せにせず組織標準と継続検証を組み合わせる。

8.5.18 APIセキュリティとREST【2026-10-28 読み上げ追加】

REST（Representational State Transfer）とは

- **REST**はネットワークを介したリソース操作を設計するための**アーキテクチャスタイル**。特定の認証方式や通信プロトコルそのものではない。
- RESTful APIでは、HTTPなどを用いてURIで示したリソースにアクセスし、一般に**GET（取得）／POST（作成・処理）／PUT（置換）／PATCH（部分更新）／DELETE（削除）**などを使う。
- 特徴としてクライアント・サーバーの分離、**ステートレス**なリクエスト、キャッシュ可能性、統一されたインターフェース等を意識する。
- **セキュリティの要点**：RESTを採用すること自体は安全性を保証しない。認証、リソース単位の認可、TLS、入力検証、レート制限、監視が必要。

教材のREST APIセキュリティ推奨事項

教材で読み上げられた推奨事項	狙いと実務上の補足
Webアプリ同様のセキュリティ対策をAPIにも適用する	APIを例外扱いしない。Webで利用している入力検証・監視・アクセス制御をAPIにも適切に適用。**XSS対策は出力文脈に依存**するため、APIではJSONの適切な扱い・Content-Type、入力検証、呼び出し元での安全な表示も考える
独自のセキュリティ機能を作り直さない	検証済みの認証ライブラリ、標準フレームワークを活用し、設計・実装ミスを減らす
APIキーだけの認証を高い保護が必要なAPIで安易に使わない	APIキーは呼び出し元識別や利用量制御には有用だが、**利用者本人の強固な認証・認可の代替には通常ならない**。公開読み取り専用APIでもレート制御等は要検討
認証情報・秘密鍵を平文で送らない	**HTTPS（TLS）**で通信路を保護し、鍵やトークンを適切に保存・ローテーションする。HTTP平文のまま資格情報を送信しない
HMACと安全なハッシュ関数を検討する	**HMAC-SHA-256など**は共有秘密鍵を用いたメッセージ認証で、改ざん検出とメッセージ認証に使う。SHA-2／SHA-3系を優先し、MD5やSHA-1は新規設計で避ける

重要な補足： HMACは通信内容を**暗号化しない**ため、機密性にはTLS等が必要。HMACとパスワード保存用のハッシュは用途が異なる。また「HMACが常に最も安全」「APIキーは全用途で禁止」とは断定しない。APIの要件・脅威モデルに応じて方式を選定する。

認証・認可方式：教材で言及した3種

方式	概要	CISSP上の留意点
HTTP Basic認証＋TLS	IDとパスワードをHTTP Authorizationヘッダーで送る。TLSで通信路を保護	Base64は暗号化ではない。TLS必須、資格情報漏洩・再利用のリスクに注意
OAuth 1.0a	署名を用いてAPIリクエストを認可する旧来のOAuth方式	リクエスト署名とトークンを利用。構成が複雑で、新規構築では要件に応じて現行方式を検討
OAuth 2.0	アクセストークンを使って**委任された認可**を実現するフレームワーク	**OAuth 2.0自体は認証プロトコルではなく、主目的は認可**。ユーザー認証には通常OpenID Connect（OIDC）等を組み合わせる。TLS、スコープ、トークン有効期限、適切なフローを管理

補足： 読み上げ教材では「認証オプション」として並べられたが、厳密には**Basic認証とOAuthの認可機構は役割が違う**。HTTP BasicはID・パスワードによる認証、OAuthは第三者への限定されたアクセス権委任が中心。

OWASP REST Security Cheat Sheet

OWASP REST Security Cheat Sheet：REST APIを安全に設計・実装するための、OWASPが公開する実務的なチェックリスト。代表的な観点として、HTTPS、エンドポイントごとの認可、JWT・トークンの適切な検証、入力検証、HTTPメソッドの制限、適切なステータスコード、機微情報を漏らさないエラー処理、ログと監視、レート制限、CORS設定等を扱う。

CISSPでの判断順序は、**保護対象・信頼境界を把握 → 認証と各リソースへの認可を分離して設計 → 標準の暗号・認証ライブラリを使う → テストと運用監視を継続する。**

> 進捗：今回はユーザーが読み上げたセキュアコーディング標準／型安全・ポインター・コード再利用の注意と、REST APIセキュリティ・OAuth・OWASP REST Security Cheat Sheetまでを追記。以降の教材内容は未収録。

Domain 8.5 追記｜Software-Defined Security（SDS）と評価

> **統合時の注意**：この文書はSDSについての独立した追記パート。直前に案内した2,220行版の実ファイルは今回参照できていないため、既存統合マスターを上書きしない。最新版を入手できた段階で、本文の欠落がないことを確認して追記する。

Software-Defined Security（SDS：ソフトウェア定義型セキュリティ）とは

SDSは、ファイアウォール、侵入検知・防止、セグメンテーションなどのセキュリティ機能を、専用ハードウェアの個別設定に強く依存させず、**ソフトウェアによって抽象化・集中管理・自動化して提供する考え方。**ネットワーク全体に共通のポリシーを定義し、必要な位置へ柔軟に適用・更新する。

- **コントロールプレーン（Control Plane）**：何を許可・遮断するか、どこにセキュリティ機能を配置するかという方針・制御を担う。
- **データプレーン（Data Plane）**：実際の通信を転送・遮断・検査する処理を担う。
- **ビルディングブロック（Security Building Blocks）**：FW、IDS/IPS、WAF、アクセス制御、監視、ネットワーク分離など、組み合わせて利用できる防御機能の構成要素。これらをポリシーに従って柔軟に配置・変更する。

※ **SDN（Software-Defined Networking）**はネットワーク制御のソフトウェア化、**SDS**はセキュリティ機能・ポリシー適用のソフトウェア化に重点を置く。関連・連携するが同義ではない。

教材読み上げの要旨：SDSの主な利点

利点	内容・理解のポイント
① 制御の一元化	ネットワークセグメントを横断してセキュリティポリシーを集中管理し、機器ごとの管理負荷・設定オーバーヘッドを抑える。企業全体の設定変更を迅速に反映できる。
② 仮想セキュリティアプライアンスへの移行	物理専用装置だけに頼らず、仮想FW等をソフトウェアとして配備し、特定ハードウェアへの依存を減らす。
③ ポリシーの俊敏性・適応性	全社共通ポリシーから個別のセグメント・ワークロード向けポリシーまで、状況に応じてオンデマンドで変更・適用できる。
④ 柔軟な仮想オーバーレイ・ネットワーク共有	論理的なネットワーク分離・接続を柔軟に設計できる。教材では旧来のMPLS等への依存を減らし得る利点が挙げられている。**ただしSDSがMPLSを必ず不要にするわけではない。**

利点	内容・理解のポイント
⑤ 予測的な脅威対応	人間による分析・知見と機械学習を組み合わせ、脅威インテリジェンスを事後対応に加え予兆検知・予測・適応的な対策に活用する。**予測が必ず正確になるという意味ではない。**
⑥ セキュリティのビルディングブロックを俊敏に制御	FW、IDS/IPS、セグメンテーション、監視などを必要な場所に構成・再配置・自動設定しやすくする。ネットワーク変更や脅威発見に応じて防御を組み替えられる。

具体例

ある部署でランサムウェア感染の兆候を検知した場合、中央の管理基盤から当該ワークロードの通信を制限し、セグメント分離ルールを更新し、監視・検査を強化する。物理FWを一台ずつ手作業で設定する方式より、統一した方針の反映と変更を迅速化できる可能性がある。

CISSP向け：利点と同時に押さえるリスク（理解のための補足）

- **集中管理のリスク**：管理コントローラーが侵害されると、広範なルール改ざん・誤配布が起こり得る。可用性設計、MFA、最小権限、管理APIの保護、監査、変更承認を行う。
- **自動化のリスク**：設定ミスが広範囲へ高速展開され得る。変更前検証、段階的配備、ロールバックが重要。
- **仮想化≠自動的な安全性**：仮想アプライアンスも脆弱性・設定不備・運用上のリスクを持つ。
- **SDNとの混同**：SDNはネットワークのプログラム可能な制御、SDSはセキュリティコントロールの抽象化・オーケストレーションという区別を押さえる。

試験直前の識別ポイント

「集中ポリシー管理」「専用ハードウェア依存の低減」「仮想セキュリティ機能」「オンデマンドの動的変更」「防御機能のオーケストレーション」→ SDS。 ただし、集中管理の管理プレーン自体も重要な保護対象となる。

出典・整理上の区別

- **教材読み上げ由来**：中央制御、管理負荷削減、仮想アプライアンス、動的ポリシー、仮想オーバーレイ、MPLSへの依存低減、機械学習と人間の知見による予測型の対応、ビルディングブロックの俊敏な制御。
- **理解用補足／外部確認**：SDSとSDNの概念上の区別、コントロール／データプレーン、集中コントローラーの保護、MPLS廃止を必然としない注意点。
- 参考：NSA *Managing Risk from Software Defined Networking Controllers*（2023）、Cisco *Software-Defined Networking*、TSO-DSO Cooperation Platform *Software Defined Security*。

SDS革命（SDS Revolution）：中核概念の拡張

教材読み上げの要旨

SDSは単なるネットワーク防御装置のソフトウェア化にとどまらず、**アイデンティティを基軸にした、きめ細かく、先回りし、迅速に適応するセキュリティ**へ範囲を拡大している。教材に挙げられた主張を以下に整理する。

教材の主張	意味・具体的な理解

教材の主張	意味・具体的な理解
① アイデンティティ管理が不可欠	ポリシーを適切に適用するには、アクセス主体の識別・認証・管理・追跡が重要。人間の利用者だけでなく、サービス、アプリケーション、デバイスなどのエンティティも対象とする。
② 防御をより積極的・予測的にする	事後対応だけでなく、観測と脅威インテリジェンスから学び、検知・判断・対処を高速化する。防御側も攻撃者の変化に適応する。
③ 組織全体の知識と能力を高める	セキュリティチームの専門性だけでなく、関係者の情報保証・情報資産保護の理解を底上げする。ツールだけで安全は達成できない。
④ システムは俊敏・柔軟・応答的である	状況や脅威の変化に合わせ、設定・保護範囲・コントロールを迅速に変更できることが望ましい。
⑤ アクセス管理を保護対象の細粒度へ移す	サーバー全体への広範なアクセス権より、アプリ、ワークロード、データ、操作単位の必要最小限の権限を重視する。一般利用者への不要なサーバー／システムレベル権限を避ける。
⑥ ゼロトラストとの相互関係	ゼロトラストの継続的な検証・最小権限・個別資源保護の考え方が、SDSによる動的なポリシー適用を支え、逆にSDSのソフトウェア制御がゼロトラストの実装を支援する。

用語を混同しないための補足

- **アイデンティティ（Identity）**：人間だけではなく、端末・サービス・ワークロードなどにも割り当てられる識別情報。識別、認証、認可、監査・追跡は異なる活動。
- **細粒度アクセス制御（Fine-Grained Access Control）**：対象資源・操作・条件を細かく指定する。サーバー単位の一括許可より、必要なデータやAPI操作のみを許す設計が例となる。
- **ゼロトラスト（Zero Trust）**：ネットワーク内部にいるだけで信頼しないという原則。**SDSとゼロトラストは同義ではなく、両者の採用が常に技術的な必須条件というわけでもない。** 教材の「相互に必要・可能にする」は、相補的なアーキテクチャ上の関係として読む。
- **教育・人による判断**：機械学習や自動化を使っても、ポリシー決定、説明責任、例外、リスク受容は人と組織の管理を要する。

理解用の例

社内の給与サービスについて、従来の「社内ネットワークに接続できれば給与サーバーへ広く接続可能」という設計から、**ユーザー／サービスID、端末状態、要求するAPI、閲覧・更新するデータに応じた認証・認可**へ移行する。SDSの集中制御・自動化によってポリシー変更を各ワークロードへ展開し、監視結果に応じてアクセスを再評価できる。

Software-Defined Security とアセスメント：2つの問い

教材で明示された基本質問

質問	何を評価するか	確認できる証拠例（補足）

質問	何を評価するか	確認できる証拠例（補足）
Q1. 組織が依存するシステムは、コンプライアンス要求と現在の脅威環境に対応するよう、適切に運用されているか？	**システム全体の運用・リスク・遵守の妥当性**。規制や組織ポリシーを満たすだけでなく、現実の脅威に対処できるか。	リスクアセスメント、構成ベースライン、法令・標準への適合評価、アクセスレビュー、脆弱性管理、監視記録。
Q2. 導入したSDSおよびその他のセキュリティプロセスは正しく機能しているか？	**採用したセキュリティコントロールの実効性**。設計・設定どおりに動くだけでなく、期待する防御を実際に発揮しているか。	ポリシー適用テスト、変更後の回帰テスト、認証・認可の否定テスト、ログ監査、検知・隔離の訓練と結果、継続的な統制評価。

核心となる区別：Q1は『守るべきシステムが適切に運用されているか』、Q2は『守るための仕組みが実際に機能しているか』。 関連するが、片方が満たされても他方が自動的に成立するわけではない。

CISSP試験の判断軸（補足）

セキュリティ対策の導入完了だけでは不十分。**要求事項・現実の脅威・実際の運用状況**を照らして評価し、**コントロールの有効性をテスト・監査して、残余リスクを権限者へ報告する**。継続的な監視と改善が必要。

Domain 8 まとめ：締めくくり

Domain 8では、**開発ライフサイクル全体で安全性を組み込むこと、コード／API／再利用部品／開発環境を管理すること、テストと評価により統制の実効性を確認すること**が一貫した判断軸になる。SDSはその延長線上にある、柔軟で動的なセキュリティ制御の考え方として理解する。

最終暗記フック：SDS＝集中制御＋仮想化＋動的ポリシー／SDS革命＝ID＋積極的防御＋教育＋俊敏性＋細粒度認可＋ゼロトラスト／評価＝『運用は適切か』『統制は機能するか』。

本節の情報の区分

- **読み上げ由来**：6つの中核主張と2つの評価質問。
- **理解・試験対策のための補足**：サービスID、細粒度制御の例、ゼロトラストとの厳密な区別、証拠やテスト例、Domain 8総括。